

SABLM-VD: Vulnerability detection with a semantic-aware binary language model

Qinghao Li, Tieming Liu, Wei Liu, Yonghe Tang, Chunling Liu, Weiyu Dong^{ID*}

Information Engineering University, Zhengzhou 450000, China

ARTICLE INFO

Dataset link: <https://github.com/luckylili/SABLM-VD>

Keywords:
Vulnerability detection
Binary analysis
Deep learning
Language model

ABSTRACT

Context: Static detection of binary code vulnerabilities based on deep learning is an important research field in computer security.

Objective: In existing methods, those mainly based on code similarity often focus on overall similarity while neglecting crucial and subtle semantic differences in the code, leading to potential false positives. Meanwhile, methods mainly based on vulnerability patterns still face challenges in learning semantic features.

Method: To address the above problems, we conduct research on x86-64 and ARM architecture binaries and propose a hybrid-granularity assembly language tokenization method. Then, we propose a BERT-based semantic-aware binary language model, SABLM. It is a solution that simultaneously embeds the data transfer semantics, arithmetic logic semantics, and control flow of assembly instructions into a BERT-based language model, effectively perceiving the semantics of assembly code. Based on the semantic feature representation of assembly code by SABLM, we incorporate pseudocode features and construct a cross-architecture vulnerability detection framework, SABLM-VD.

Results: We evaluate SABLM-VD on the NVD dataset and the SARD dataset. The results show that SABLM-VD outperforms the state-of-the-art baseline methods in terms of F1-score, precision, recall, and accuracy, with SABLM-VD achieving F1-scores of 79.33% on NVD (Mixed), 100.00% on SARD (x86-64), and 100.00% on SARD (ARM). Ablation studies and component analysis demonstrate the effectiveness of each component of SABLM-VD. Visualizations and real-world applications further confirm the advantages of SABLM-VD.

Conclusion: Our research indicates that SABLM-VD, which is based on the semantic-aware binary language model SABLM and pseudocode features, can effectively detect binary code vulnerabilities, warranting further research in this direction.

1. Introduction

Code vulnerabilities are a major threat to computer information systems. In recent years, with the explosive development of the open source community, the unreliable third-party code has accounted for an increasing proportion of daily software development, which has further exacerbated the spread of code vulnerabilities. In OSSRA 2024 [1], Synopsys' security team anonymously analyzed 1067 codebases in 17 industries and found that 84% of the codebases were at risk of vulnerabilities. Faced with such a large number of codes with vulnerability risks and executable programs built from them, traditional code audit and program analysis by domain experts requires high labor costs and is difficult to adapt to the massive growth of code and executable program scale. In contrast, with the continuous breakthroughs in the field of deep learning, vulnerability detection methods based on deep learning can meet the needs of large-scale code audit and program analysis with

their fast and low-cost characteristics, and have become a research hotspot in the field of computer information security.

In general, code vulnerabilities are detected in two forms: source code and binary code. In terms of source code, vulnerability detection based on deep learning research [2–6] showed positive and effective results. In terms of binary code, limited by the inherent nature of binary code, the research progress of vulnerability detection based on deep learning is relatively slow. Specifically, the binary code is compiled from the source code. For the same C/C++ source code, different versions of different compilers (such as GCC, Clang) under different architectures (such as x86-64, ARM) and according to different optimization levels (such as O0, O1, O2, O3) will generate different binary codes. Moreover, compile-irrelevant information such as variable types, variable names, and comments in the source code will also be discarded during the compilation process. Therefore, the

* Corresponding author.

E-mail address: dongxinbaoer@163.com (W. Dong).

<https://doi.org/10.1016/j.infsof.2025.107959>

Received 23 June 2025; Received in revised form 29 October 2025; Accepted 31 October 2025

Available online 12 November 2025

0950-5849/© 2025 Elsevier B.V. All rights reserved, including those for text and data mining, AI training, and similar technologies.

complexity and difficulty of binary code analysis are usually higher than that of source code.

In fact, in order to detect binary code vulnerabilities quickly and effectively, various methods based on deep learning have been continuously proposed in recent years. The method [7] extracts the Attributed Control Flow Graph (ACFG) of binary code and performs graph embedding, and then detects vulnerabilities by comparing it with a vulnerability database using Locality Sensitive Hashing (LSH). The method [8] extracts and embeds the Labeled Semantic Flow Graph (LSFG) of binary code, and subsequently utilizes cosine similarity to detect cross-platform similar vulnerabilities. The method [9] embeds LSFG and then employs a Siamese network combined with simulation to detect vulnerabilities. The method [10] designs Context Window Prediction (CWP) and Def-Use Prediction (DUP) pre-training based on BERT for neural networks to capture assembly language patterns. The method [11] conducts pre-training specifically for jump instructions in assembly code, aiming to enable the neural network detection model to fully understand the control flow characteristics of assembly instructions. The method [12] detects suspected vulnerabilities across architectures through an entropy-based adapter and intermediate representation model. Methods [13,14] all detect suspected vulnerabilities from the perspective of graph features of code.

However, the aforementioned deep learning-based binary code vulnerability detection methods still face several challenges: (1) **The similarity comparison is prone to overlooking crucial and subtle semantic differences.** Both graph-based methods [7–9] and sequence-based methods [10–12] conduct vulnerability detection on the basis of code cloning detection, which involves comparing the embedding vectors of target binary code with those of vulnerability database samples for similarity. These methods focus more on overall similarity than local subtle semantic differences in the code, and determine comparison results through threshold setting, which makes it difficult to identify vulnerabilities hidden within character-level differences, such as boundary check errors, and may lead to misclassification of benign code (with minor patches) that is overall similar to vulnerability database samples. (2) **The learning and capture of vulnerability patterns are still not comprehensive and accurate enough.** Methods [7,8,14] utilize Control Flow Graph (CFG)-dominated graph structural features as the feature representation of vulnerability samples, neglecting the semantic features of data flow and arithmetic logic operations in assembly code. The method [13] extracts Abstract Syntax Tree (AST) and Program Dependence Graph (PDG) from the pseudocode of assembly code as feature representations, using pseudocode to bridge the gap between binary code and source code, but discards the sequential features of original assembly instructions, which is negative for the detection model to learn the vulnerability patterns of binary code.

To address these challenges, we propose a Vulnerability Detection framework with a Semantics-Aware Binary Language Model, SABLM-VD.

Specifically, for the two challenges mentioned, we believe that it is necessary to strengthen the understanding of the semantics of the binary code, especially the understanding of key assembly instructions related to semantics and their contextual relationships. By focusing on the locations and meanings of the key instructions, the detection model will not ignore small differences related to vulnerability semantics and will be able to learn and capture vulnerability patterns more efficiently. Considering the cross-architecture difference problem of binary code, we conduct the study under both x86-64 and ARM architectures. We concatenate assembly instructions from both architectures on a per-line basis to form assembly sequences, and design a hybrid-granularity assembly language tokenization method to tokenize assembly sequences. Based on the token sequence, we propose SABLM, a semantic-aware binary language model based on BERT [15], which aims to generate semantic embeddings of assembly instructions through semantic-level pre-training. For this purpose, we design three assembly instruction pre-training tasks based on the nature and instances of the x86-64

and ARM architecture instruction sets: Data Transfer Prediction (DTP), Arithmetic Logic Prediction (ALP), and Control Flow Prediction (CFP), to train SABLM to capture data transfer semantics, arithmetic logic semantics, and control flow features of assembly instructions. The DTP, ALP, and CFP tasks, by differentially and concurrently masking the destination operand, the first source operand (if any), and the second source operand (if any) of specific instructions, prompt the model to understand the relationships between specific operands, the relationships between specific opcodes and operands, as well as the meanings and roles of specific operands in the global context, and predict the masked positions, thereby learning more fine-grained and comprehensive semantics.

After pre-training, SABLM can provide high-quality embeddings for assembly instructions. On the basis of SABLM embedding, to further expand the code feature surface, we also introduce pseudocode features. We combine the SABLM embeddings of assembly instructions with the embeddings of subword sequence-level pseudocode and construct a cross-architecture vulnerability detection framework through the effective classification of MLP, SABLM-VD.

We have implemented SABLM-VD. To evaluate the performance of SABLM-VD, we constructed two datasets, the SARD dataset and the NVD dataset. Specifically, we compiled the source codes from SARD [16] using GCC on both ARM and x86-64 architectures to obtain binary codes as the SARD dataset. We collected 450 CVE [17] vulnerability/patch source codes from 8 representative open-source projects and compiled them under two architectures (x86-64 and ARM), two compilers (GCC and Clang), and four optimization levels (O0, O1, O2, and O3) to obtain binary codes as the NVD dataset. We compared SABLM-VD with 10 state-of-the-art software vulnerability detection methods under the SARD dataset and the NVD dataset. The experimental results demonstrate that our proposed method outperforms the current state-of-the-art baseline methods.

In summary, the major contributions of our work in this paper are as follows:

- (1) We propose a hybrid-granularity assembly language tokenization method that can effectively preserve assembly instruction information and solve the OOV problem.
- (2) We propose a novel BERT-based semantic-aware binary language model, SABLM, which is a solution that simultaneously embeds the data transfer semantics, arithmetic logic semantics, and control flow of assembly instructions into BERT.
- (3) We propose a binary code vulnerability detection framework, SABLM-VD, which learns the SABLM embedding of assembly instructions and the pseudocode embedding. We conduct comprehensive experiments to evaluate SABLM-VD and validate the technical designs in SABLM-VD. The evaluation results demonstrate that SABLM-VD outperforms all baselines, and the technical solutions in SABLM-VD are effective. In addition, we publicly released our code and configuration requirements to facilitate future research: <https://github.com/luckyli/SABLM-VD>.

2. Related work

In recent years, the research on binary code vulnerability detection based on deep learning has made a lot of progress. This section focuses on the important parts: (1) feature representation of binary code, (2) feature learning and vulnerability detection of binary code.

2.1. Feature representation of binary code

Binary code includes machine code, assembly code, and pseudocode. Machine code consists of executable binary digits but lacks intuitive semantics, hindering analysis. Assembly code represents machine code in assembly language, typically derived using disassembly algorithms from reverse engineering tools like IDA [18]. It retains

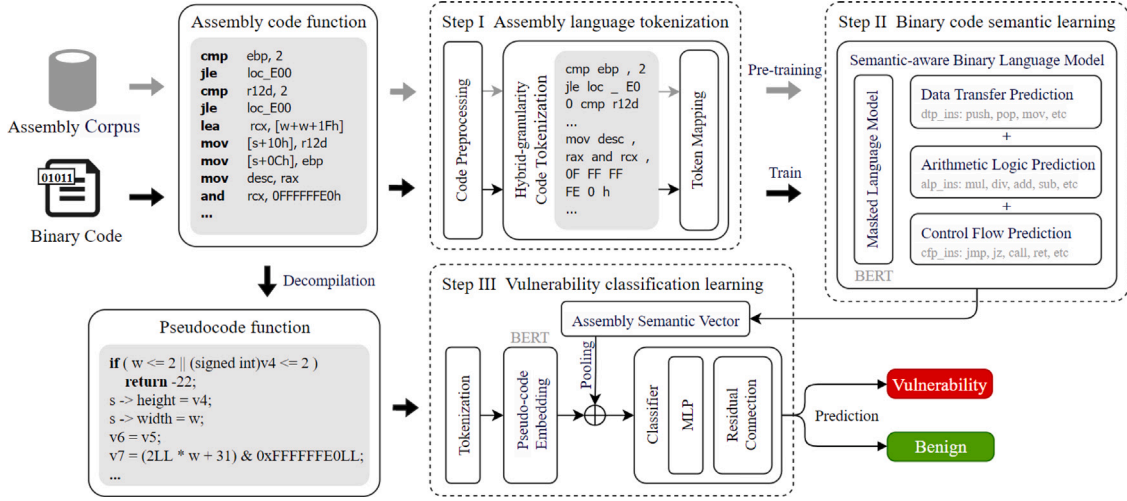


Fig. 1. The framework of SABLM-VD. The gray arrow part is the pre-training flow, and the black arrow part is the training flow.

hardware-specific details and is the most widely analyzed form. Pseudocode, generated by decompiling assembly code, features high-level language syntax.

Machine code feature representations include bytecode images, byte sequences, etc. The method [19,20] performs binary analysis by converting machine code into bytecode images. The method [21] uses the machine code as a feature representation in the form of hexadecimal byte sequences.

Assembly code feature representations include assembly instruction sequences, CFGs, etc. The methods [22–24] use assembly instruction sequences as feature representations, whereas methods [12,25–27] extract CFGs from assembly code to analyze control flow characteristics.

Pseudocode feature representations include pseudocode sequences, ASTs, PDGs, etc. The method [13] uses pseudocode sequences and the ASTs and PDGs extracted therefrom as code feature representations to counteract the effects of compilation diversity.

2.2. Feature learning and vulnerability detection of binary code

To effectively learn binary feature representations for vulnerability detection, it is necessary to design different neural network models based on the type of feature representation. In recent years, the RNN family, GNN family, CNN family, Transformer, and its derived networks have all been applied to feature learning and vulnerability detection. HAN-BSVD [23] utilizes BGRU to generate embeddings for assembly instruction sequences. BinVulDet [13] feeds sliced code from ASTs and PDGs into a BLSTM combined with self-attention layers for learning. VulHawk [12] and VulAnalyzer [14] learn CFG features through GCNs. PalmTree [10], jTrans [11], Trex [24], and HAformer [21] all use BERT as the base model for assembly code learning. Additionally, α Diff [28], BinSeeker [9], and Asteria [29] employ a Siamese network framework to capture similarities among vulnerability samples.

However, these methods still face challenges in accurately and deeply learning the semantic features of binary code, which leads to false positives and false negatives in vulnerability detection. Therefore, this paper designs a hybrid-granularity assembly language tokenization method and proposes a BERT-based cross-architecture semantic-aware binary language model, SABLM. Through pre-training tasks DTP, ALP, and CFP, SABLM comprehends the data transfer semantics, arithmetic logic semantics, and control flow of assembly code. Finally, this paper constructs a vulnerability detection framework, SABLM-VD, by combining SABLM embeddings with pseudocode sequence features, effectively detecting vulnerabilities.

3. Proposed model

In this section, we introduce the overall framework of SABLM-VD, as shown in Fig. 1. SABLM-VD consists of three main parts: (1) Assembly language tokenization, which aims to map assembly code into a token sequence using a hybrid-granularity tokenization method. (2) Binary code semantic learning, which aims to enable SABLM to learn and understand the data transfer semantics, arithmetic logic semantics, and control flow of assembly code through pre-training tasks, including DTP, ALP, and CFP. (3) Vulnerability classification learning, which aims to combine the assembly semantic vectors output by SABLM with pseudocode embedding vectors to form the final vulnerability classification model.

3.1. Problem formulation

Determining whether a binary code is a vulnerability code is a binary classification problem. Lipp et al. found that 92% of vulnerabilities reside within the same function, establishing that function granularity is an appropriate granularity for detection [30]. In this paper, we detect binary codes at the function granularity, aiming to classify functions into vulnerable and benign categories. We denote the dataset containing vulnerable and benign samples as $S = \{x_i, y_i\}$, where x_i denotes the binary code function, and y_i denotes the ground truth label indicating whether it is a vulnerable function or a benign function, with 1 for vulnerable and 0 for benign. The objective of SABLM-VD is to learn a mapping, $f : x_i \mapsto y_i$, to predict whether a function belongs to the vulnerable class or the benign class. Typically, to measure the loss between predicted values and ground truth values, we use the cross-entropy loss function [31], formulated as follows:

$$\mathcal{L}_{CE}(\hat{y}_i, y_i) = -y_i \cdot \log(\hat{y}_i) + (y_i - 1) \cdot \log(1 - \hat{y}_i) \quad (1)$$

where y_i is the ground truth values and $\hat{y}_i$ is the predicted output of the model.

3.2. Assembly language tokenization

This stage takes the assembly code function as input to generate the corresponding hybrid-granularity token sequence representations. In this section, we detail how to build token sequence representations from assembly code functions, which consists of 2 parts: code preprocessing and hybrid-granularity code tokenization.

3.2.1. Code preprocessing

The assembly code function is a segment of binary code in assembly language form at the function-level, obtained from a binary file. Specifically, we use the general high-performance reverse engineering tool IDA [18] to extract the machine code from the binary file and disassemble it into assembly code. We intercept the assembly code by function, retaining only the assembly instruction content. It should be noted that the assembly code generated by IDA disassembly is in Intel format.

Code preprocessing aims to reduce redundant information in assembly code functions while preserving their informational integrity, thereby reducing the occupancy of invalid tokens. We first split the assembly code function by lines, with each line containing a single assembly instruction. Then, we remove redundant non-visible symbols from assembly instructions without compromising the separation of opcodes and operands, as detailed below:

- (1) Replace all occurrences of tabs and carriage returns with spaces.
- (2) Remove leading and trailing spaces from each line.
- (3) Collapse consecutive spaces into a single space.
- (4) Remove spaces immediately following commas.

3.2.2. Hybrid-granularity code tokenization

Code tokenization involves code splitting and token mapping, where the vocabulary is typically defined by splitting. The constraint of vocabulary size (e.g., 50,265 in RoBERTa) leads to varying degrees of the out-of-vocabulary (OOV) problem, where unseen tokens can adversely affect model performance. jTrans [11] avoids splitting opcodes/operands, replacing memory operand offsets with CONST, shifts with var_XXX, and jump addresses with JUMP_ADDR_X. This method reduces vocabulary size but removes explicit addresses. CodeBERT [32] and UniXcoder [33] split rare words into subwords and reorganize them to preserve common words and mitigate OOV.

This paper proposes a hybrid-granularity code tokenization method, aiming to alleviate the OOV problem while effectively splitting the basic units of code through appropriate granularity. It also avoids generating overly long token sequences (overly long token sequences often exceed the input limits of fixed-length neural network models, leading to unavoidable information truncation), thereby enabling neural network models to learn code information more accurately and comprehensibly. Specifically, we adopt a code splitting strategy that preserves word-level granularity for opcodes and separating symbols, while splitting operands into subword-level granularity, to construct the vocabulary and complete the mapping from assembly language to token sequences.

We use code preprocessing to extract all assembly code lines $\mathbb{L}_{asm}$ from the assembly function $\mathbb{F}_{asm}$, and then apply Algorithm 1 for hybrid-granularity code tokenization, completing the mapping from $\mathbb{L}_{asm}$ to the token sequence $\mathbb{T}_{line}$. In Algorithm 1, we define the initial, further, and final word sequences as $\mathbb{W}_{ini}$, $\mathbb{W}_{fur}$, and $\mathbb{W}_{fin}$, respectively, to store the split words at each phase.

After the first loop in Algorithm 1, we split $\mathbb{L}_{asm}$ using non-alphanumeric characters to obtain $\mathbb{W}_{ini}$. In assembly code, non-alphanumeric characters typically include spaces, commas (“,”), square brackets (“[]”), plus signs (“+”), etc. Splitting by these characters not only separates word-level opcodes and operands but also isolates the base address and offset within composite operands (e.g., the instruction `mov edx, [r13+28h]` would be split into “mov”, “”, “edx”, “”, “[”, “r13”, “+”, “28h”, “]”).

In the second loop, we split $\mathbb{W}_{ini}$ using 0x and 0000 to obtain $\mathbb{W}_{fur}$. This is because hexadecimal numbers in ARM architecture assembly code typically start with 0x, and 0000 is a set of digits that frequently appears in the first four positions of an 8-character hexadecimal base address.

In the third loop, we split all hexadecimal data in $\mathbb{W}_{fur}$ into 2-character units to obtain $\mathbb{W}_{fin}$, refining handling of immediates/addresses/offsets. While the second loop processed the 0x

Algorithm 1 Hybrid-granularity code tokenization.

Input: $\mathbb{L}_{asm}$: a assembly code line,
 $\mathbb{W}_{ini}, \mathbb{W}_{fur}, \mathbb{W}_{fin}$: respectively the initial, further, and final splitted word sequence, initialized as null,
 $\mathbb{V}$: the vocabulary for token mapping,
 P_{split} : the pattern used for splitting,
 P_{match} : the pattern used for matching,
 Σ : the whole character set.

Output: $\mathbb{T}_{line}$: the token sequence of the assembly code line.

HybridToken($\mathbb{L}_{asm}$) :

```

for  $c \in \Sigma$  do
  if  $c \notin [a-zA-Z0-9]$  then
     $P_{split} \leftarrow c$  ; // Split with non-alphanumeric characters
  end
end
 $\mathbb{W}_{ini} \leftarrow \text{Split}(P_{split}, \mathbb{L}_{asm})$ ;
for  $w_{ini} \in \mathbb{W}_{ini}$  do
  if  $0x \in w_{ini}$  or  $0000 \in w_{ini}$  then
     $P_{split} \leftarrow 0x$  or  $0000$  ;
     $\mathbb{W}_{fur}' \leftarrow \text{Split}(P_{split}, w_{ini})$ ;
     $\mathbb{W}_{fur} \leftarrow \text{Append}(\mathbb{W}_{fur}')$ ;
  end
else
   $\mathbb{W}_{fur} \leftarrow \text{Append}(w_{ini})$ ;
end
// Split the words containing 0x or 0000
end
 $P_{match} \leftarrow [0-9A-F]^+ ;$  // Match the hexadecimal number
for  $w_{fur} \in \mathbb{W}_{fur}$  do
  if  $\text{Match}(P_{match}, w_{fur})$  and  $\text{Endswith}(w_{fur}) = h$  then
     $hex \leftarrow w_{fur}$  ;
    for  $2c \in hex$  do
       $\mathbb{W}_{fin} \leftarrow \text{Append}(2c)$ ;
      // Split the hexadecimal part by 2-character units
    end
     $\mathbb{W}_{fin} \leftarrow \text{Append}(h)$ ;
  end
else if  $\text{Match}(P_{match}, w_{fur})$  then
  for  $2c \in w_{fur}$  do
     $\mathbb{W}_{fin} \leftarrow \text{Append}(2c)$ ;
  end
end
else
   $\mathbb{W}_{fin} \leftarrow \text{Append}(w_{fur})$ ;
end
end
 $\mathbb{T}_{line} \leftarrow \text{Map}(\mathbb{W}_{fin}, \mathbb{V})$ 
return  $\mathbb{T}_{line}$ 

```

prefix of ARM, this loop additionally addresses the h suffix of x86-64. We thus distinguish hexadecimal numbers by suffix presence and uniformly split them into 2-character units. Given the variable length of hexadecimal numbers, the optimal interval 2 was experimentally determined (Section 5.3). Finally, $\mathbb{W}_{fin}$ is mapped to token sequence $\mathbb{T}_{line}$ via vocabulary $\mathbb{V}$.

After Algorithm 1, we concatenate all $\mathbb{W}_{fin}$ and $\mathbb{T}_{line}$ corresponding to $\mathbb{L}_{asm}$ in the assembly code function $\mathbb{F}_{asm}$ according to row order, forming the function-splitted word sequence $\mathbb{W}_{asm}$ and function token sequence $\mathbb{T}_{asm}$. Before inputting into the neural network, we remove all spaces in the former and all space tokens in the latter.

The vocabulary $\mathbb{V}$ used for token mapping is also generated by Algorithm 1. Specifically, we first constructed an assembly corpus that includes both 336,939 x86-64 assembly code functions and 57,935 ARM assembly code functions (details can be seen in subsection (3) of

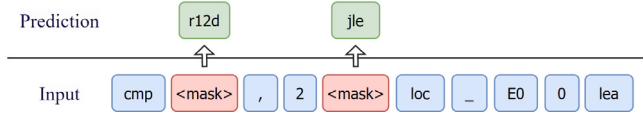


Fig. 2. Masked language model.

Section 4.3). Then, we applied the splitting part of Algorithm 1 (the three loop structures) to this corpus, producing a hybrid-granularity split of all the corpus data. We counted the frequency of all split words and mapped them to sequence numbers in descending order of frequency, thereby constructing the vocabulary $\mathbb{V}$.

3.3. Binary code semantic learning

The input for the binary code semantic learning stage is a hybrid-granularity token sequence representation, aiming to learn and understand the data transfer semantics, arithmetic logic semantics, and control flow of assembly code through SABLM that includes pre-training tasks DTP, ALP, and CFP. Specifically, we use the split word sequence $\mathbb{W}_{asm}$ and the token sequence $\mathbb{T}_{asm}$ as inputs, and learn various semantic features of assembly code through BERT-based pre-training tasks. To optimally learn multiple features, it is necessary to tailor pre-training tasks for different types of assembly instructions. Therefore, on the basis of the original masked language model (MLM), we propose the pre-training tasks of data transfer prediction, arithmetic logic prediction, and control flow prediction, as shown in Step II of Fig. 1. The pre-training process of SABLM is divided into 2 steps. Firstly, we implement the MLM on the assembly corpus. Then, on the weights of the best-performing MLM, we simultaneously implement the DTP, ALP, and CFP to obtain the final SABLM. In terms of understanding important features of code, the effectiveness of BERT with a bidirectional attention mechanism has been demonstrated in multiple studies [5,10–12,34].

It should be noted that the DTP, ALP, and CFP tasks proposed are inspired by JTP of jTrans [11] and CWP/DUP of PalmTree [10], but differ significantly. JTP only handles operand positions in j/J-prefixed control instructions, ignoring call/ret impacts and lacking cross-architecture applicability (e.g., jump instructions in ARM start with b/B). CWP/DUP focus solely on line-granularity context correlation and require Binary Ninja [35] to extract control pairs and definition-use pairs. In contrast, our tasks: (1) Require no external tools; (2) Cover data/arithmetic/control instructions at hybrid-granularity across x86-64/ARM; (3) Employ different and concurrent masking of destination/first/second source operands to train the model to understand fine-grained and comprehensive semantics (operand-operand relationships, opcode-operand relationships, global roles of specific operands). (The quantitative comparison on the vulnerability detection performance of DTP+ALP+CFP versus JTP, CWP+DUP can be seen in Section 5.4.)

3.3.1. Masked language model

We introduce MLM to initially understand the relationships between different assembly words and form a basic binary language model. MLM was first proposed by BERT [15], which predicts the masked tokens based on their surrounding context tokens to optimize model parameters. In this paper, we randomly select 15% of the tokens in $\mathbb{T}_{asm}$ for replacement. The selected tokens are replaced with the <mask> token. Fig. 2 shows an example, where the red, green, and blue boxes represent the <mask> tokens, prediction results, and context information, respectively. In Fig. 2, the register r12d and the opcode jle are masked as <mask>.

During the pre-training process, we feed the final hidden states corresponding to the masked tokens into the $Softmax(\cdot)$ output layer

Table 1
Data transfer instructions.

Opcode function	x86-64	ARM
Stack push	push, pushfq	PUSH
Stack pop	pop, popfq	POP
Data movement	mov	MOV
Other memory access	–	LDM, STM, LDR, STR

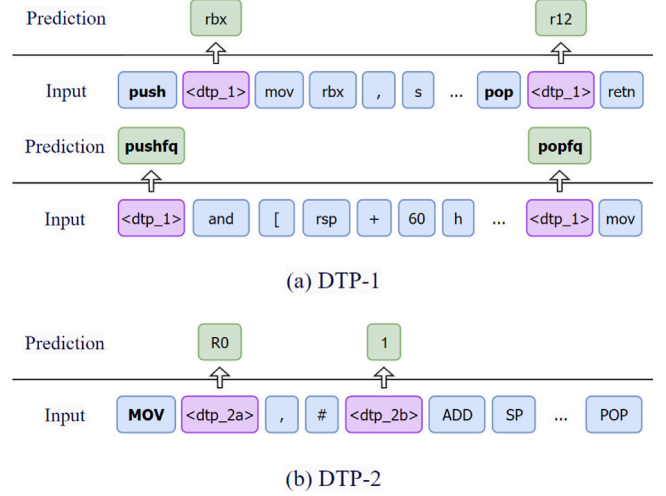


Fig. 3. Data transfer prediction.

over the vocabulary $\mathbb{V}$ to predict the probabilities of these tokens. The loss function for MLM also adopts CE, specifically as follows:

$$\mathcal{L}_{MLM}(\theta_1) = - \sum_{i \in M} \log P(y_i = \hat{y}_i | \theta_1) \quad (2)$$

where y is the ground truth values and $\hat{y}$ is the prediction, both ranging within the size of $\mathbb{V}$, M denotes the masked token set, and θ_1 denotes the attention head parameter of the MLM.

3.3.2. Data transfer prediction

We propose the DTP pre-training task, which associates opcodes related to data transfer with the semantics of their operands, enabling SABLM to understand the data transfer semantics. Specifically, in the assembly language, we use the x86-64 official documentation [36–38] and the ARM official documentation [39] as the bases for assembly instruction definitions under these two architectures, respectively. From these, we filter out all instructions related to data transfer, including instructions for stack push, stack pop, data movement, memory access, and address transfer. Then, based on the assembly corpus, we exclude the instruction types that appear less than 100 times, ultimately forming a list of data transfer instructions as shown in Table 1. Depending on the number of operands associated with the instructions, we categorize all instructions under the two architectures into two types: DTP-1 and DTP-2. We designed corresponding DTP tokens and masking forms, and extended the form of instruction opcodes, as shown in Table 2.

DTP-1: Involves at most 1 composite operand, including stack push and stack pop instructions, with opcodes push, pop, pushfq, popfq, PUSH, POP, using the token <mask>. We train the model to understand the semantics of stack push and stack pop by masking the content of the source operands (excluding symbols) for push, pop, PUSH, POP, and the instructions pushfq, popfq themselves. (This is achieved by locating the corresponding tokens in $\mathbb{T}_{asm}$ using $\mathbb{W}_{asm}$ and replacing them with the <mask> token, and the same applies to subsequent operations.) An example is shown in Fig. 3(a). Under the x86-64 architecture, the operand contents of the opcodes push, pop and the opcodes pushfq, popfq themselves are masked as <mask>.

Table 2

DTP, ALP, CFP tokens, and instruction masking types. The lower-case opcode is the x86-64 instruction opcode, and the upper-case opcode is the ARM instruction opcode. The instruction extension is in the form of appending additional suffixes to the original opcode.

Type	Token	Instruction masking	Instruction extension
DTP-1	<dtp_1>	push/pop/PUSH/POP <dtp_1> pushfq/popfq → <dtp_1>	PUSH/POP+.W -
DTP-2	<dtp_2a>, <dtp_2b>	mov/MOV/LDM/STM/LDR/STR <dtp_2a>, <dtp_2b>	MOV+S/.W, STR+D/D.W, LDM/STM/LDR+.W
ALP-1	<alp_1>	inc/mul/div/ldiv/not<alp_1> (MOV OPE_1, OPE_2,) LSL/LSR <alp_1>	- -
ALP-2	<alp_2a>, <alp_2b>	add/adc/sub/sbb/imul/cmp/and/or/xor/shl/rol/rcr/MVN/CMP <alp_2a>, <alp_2b>	MVM+S/.W, CMP+EQ/.W
ALP-3	<alp_3a>, <alp_3b>, <alp_3c>	ADD/ADC/SUB/SBC/RSC/MUL/FMULD/FDIVS/AND/ORR/EOR/ASRS <alp_3a>, <alp_3b>, <alp_3c>	ADD/SUB/AND/ORR/EOR +S/.W, ADC/SBC+S
CFP-1	<cfp_1>	jmp/jz/js/jns/call/B/BEQ/BNE/BL/BLX <cfp_1> ret = <cfp_1>	B/BEQ/BNE/BL+.W -

DTP-2: Involves 2 composite operands, including data movement and other memory access instructions, with opcodes mov, MOV, LDM, STM, LDR, STR, using the token <dtp_2a> and token <dtp_2b>. We train the model to learn the semantics of general data transfer and memory access by separately masking the content (excluding symbols) of the destination and source operands. An example is shown in Fig. 3(b). Under the ARM architecture, the destination operand and the source operand content of the opcode MOV are masked as <dtp_2a> and <dtp_2b>, respectively.

To capture the semantics of data transfer while not overlooking the relationships between ordinary words, we implement DTP on the weights that have been pre-trained to optimal performance using MLM. This is a progressive pre-training process. Similarly, we predict the probabilities of DTP tokens within the vocabulary $\mathbb{V}$. The DTP loss function using CE is as follows:

$$\mathcal{L}_{DTP}(\theta_2, \theta_{dtp}) = - \sum_{i \in D} \log P(y_i = \hat{y}_i | \theta_2, \theta_{dtp}) \quad (3)$$

where y is the ground truth values and $\hat{y}$ is the prediction, both ranging within the size of $\mathbb{V}$, D denotes the masked DTP token set, θ_2 denotes the attention head parameter of the DTP, and θ_{dtp} denotes the proportion parameter of randomly masked tokens out of all maskable tokens in DTP.

3.3.3. Arithmetic logic prediction

We propose the ALP pre-training task, which associates opcodes related to arithmetic and logical operations with the semantics of their operands, enabling SABLML to understand the semantics of arithmetic and logical operations in the binary code. We adopt the same strategy as DTP to filter out all instructions related to arithmetic and logical operations under two architectures, ultimately forming a list of arithmetic logical instructions as shown in Table 3. Depending on the number of operands associated with the instructions, we categorize all instructions under the two architectures into three types: ALP-1, ALP-2, and ALP-3. We designed corresponding ALP tokens and masking forms, and extended the form of instruction opcodes, as shown in Table 2.

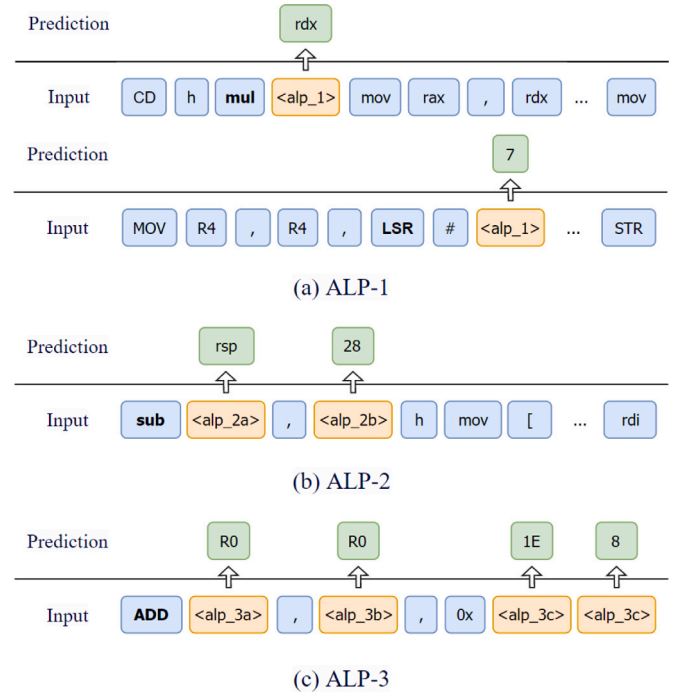
ALP-1: Involves 1 single composite operand, including certain arithmetic logic instructions for x86-64 and shift instructions for ARM, with opcodes inc, mul, div, idiv, not, LSL, LSR, using the token <alp_1>. We train the model to comprehend the semantics of arithmetic and logical operations that involve a single source operand by masking the content of the source operands (excluding symbols) for these instructions. An example is shown in Fig. 4(a), where the source operand content rdx of the opcode mul in x86-64 architecture and the source operand content 7 of the opcode LSR in ARM architecture are masked as <alp_1>.

ALP-2: Involves 2 composite operands, including arithmetic logic instructions for x86-64 and logical NOT and comparison instructions

Table 3

Arithmetic logic instructions.

Opcode function	x86-64	ARM
Addition	add, adc, inc	ADD, ADC
Subtraction	sub, sbb	SUB, SBC, RSC
Multiplication	mul, imul	MUL, FMULD
Division	div, idiv	FDIVS
Comparison	cmp	CMP
Logical AND/OR	and, or	AND, ORR
Logical NOT	not	MVN
Logical XOR	xor	EOR
Shift	shl, rol, rcr	LSL, LSR, ASRS

**Fig. 4.** Arithmetic logic prediction.

for ARM, with opcodes add, adc, sub, sbb, imul, cmp, and, or, xor, shl, rol, rcr, using the tokens <alp_2a> and <alp_2b>. We train the model to learn the semantics of arithmetic and logical operations that involve only the destination and source operands by masking the contents of these operands (excluding symbols) for these instructions. An example is shown in Fig. 4(b), where the destination

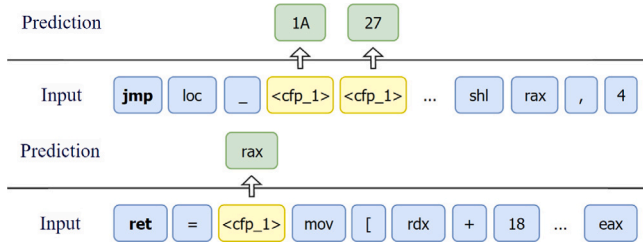


Fig. 5. Control flow prediction.

Table 4

Control flow instructions.

Opcode function	x86-64	ARM
Unconditional control transfer	jmp	B
Conditional control transfer	jz, js, jns	BEQ, BNE, BLE
Call-related	call, ret	BL, BLX

operand `rsp` and the source operand 28 of the opcode `sub` in x86-64 architecture are masked as `<alp_2a>` and `<alp_2b>`, respectively.

ALP-3: Involves 3 composite operands, exclusively including arithmetic logic instructions for the ARM architecture, with opcodes ADD, ADC, SUB, SBC, RSC, MUL, FMULD, FDIVS, AND, ORR, EOR, ASRS, using the tokens `<alp_3a>`, `<alp_3b>`, `<alp_3c>`. We train the model to comprehend the semantics of arithmetic and logical operations that involve 3 operands by masking the contents (excluding symbols) of the destination operand, the first source operand, and the second source operand for these instructions. An example is shown in Fig. 4(c), where the destination operand R0, the first source operand R0, and the second source operand contents 1E and 8 of the opcode ADD in ARM architecture are masked as `<alp_3a>`, `<alp_3b>`, and `<alp_3c>`, respectively.

To avoid ignoring other semantic information, we implement ALP alongside DTP on the same optimal MLM model weights, and similarly predict the probabilities of ALP tokens within the vocabulary $\mathbb{V}$. The ALP loss function using CE is as follows:

$$\mathcal{L}_{ALP}(\theta_3, \theta_{alp}) = - \sum_{i \in A} \log P(y_i = \hat{y}_i | \theta_3, \theta_{alp}) \quad (4)$$

where y is the ground truth values and $\hat{y}$ is the prediction, both ranging within the size of $\mathbb{V}$, A denotes the masked ALP token set, θ_3 denotes the attention head parameter of the ALP, and θ_{alp} denotes the proportion parameter of randomly masked tokens out of all maskable tokens in ALP.

3.3.4. Control flow prediction

We propose the CFP pre-training task, which associates opcodes related to control flow with the semantics of their operands, enabling SABLML to understand the semantics of control flow. We adopt the same strategy as DTP to filter out all instructions related to control flow under two architectures, ultimately forming a list of control flow instructions as shown in Table 4. All control flow instructions have only one composite operand, thus falling into a single category: CFP-1. We designed the corresponding CFP token and masking form, and extended the format of instruction opcodes, as shown in Table 2.

CFP-1: Comprises control flow instructions for x86-64 and ARM architectures, with opcodes including `jmp`, `jz`, `js`, `jns`, `call`, `B`, `BEQ`, `BEQ`, `BNE`, `BLE`, `BL`, `BLX`, using the token `<cfp_1>`. We train the model to understand the semantics of control transfers and calls by masking the content of the source operands (excluding symbols) for these instructions. It is noteworthy that, in typical assembly language code, `ret` is a standalone opcode without an operand. However, in the assembly code generated by IDA disassembly used in this paper, `ret` appears in the special form “`ret = <operand>`”, which contains

additional operand information beneficial for the model to learn the semantics of call returns. An example is shown in Fig. 5, where the source operand contents 1A and 27 of the opcode `jmp` and the source operand content `rax` of the opcode `ret` in the x86-64 architecture are masked as `<cfp_1>`.

To avoid overlooking the semantic information of data transfer and arithmetic logic, we implement CFP alongside the previous ALP and DTP on the same optimal MLM model weights, and similarly predict the probabilities of CFP tokens within the vocabulary $\mathbb{V}$. The CFP loss function using CE is as follows:

$$\mathcal{L}_{CFP}(\theta_4, \theta_{cfp}) = - \sum_{i \in C} \log P(y_i = \hat{y}_i | \theta_4, \theta_{cfp}) \quad (5)$$

where y is the ground truth values and $\hat{y}$ is the prediction, both ranging within the size of $\mathbb{V}$, C denotes the masked CFP token set, θ_4 denotes the attention head parameter of the CFP, and θ_{cfp} denotes the proportion parameter of randomly masked tokens out of all maskable tokens in CFP.

Finally, the BERT-based pre-training process of SABLML is divided into two steps.

In the first step, the MLM pre-training task is implemented on the assembly corpus. The total loss function for this stage, denoted as $\mathcal{L}_{SABLML}^{(1)}$, is as follows:

$$\mathcal{L}_{SABLML}^{(1)} = \mathcal{L}_{MLM} \quad (6)$$

In the second step, based on the weights derived from the lowest $\mathcal{L}_{MLM}$, the DTP, ALP, and CFP pre-training tasks are simultaneously implemented. The total loss function for this stage, denoted as $\mathcal{L}_{SABLML}^{(2)}$, is a combination of the loss functions of these three tasks:

$$\mathcal{L}_{SABLML}^{(2)} = \mathcal{L}_{DTP} + \mathcal{L}_{ALP} + \mathcal{L}_{CFP} \quad (7)$$

3.4. Vulnerability classification learning

The vulnerability classification learning stage aims to distinguish between vulnerable and benign binary codes by simultaneously learning both the assembly code semantic features and the pseudocode features, as shown in Step III of Fig. 1. We divide the vulnerability classification learning stage of SABLML-VD into two parts: feature vector generation and vulnerability classification.

3.4.1. Feature vector generation

For an assembly code function, the feature vectors used for SABLML-VD learning derive from two aspects: the assembly semantic vectors generated by the binary language model SABLML, and the pseudocode embeddings generated by the C language model. Specifically, these are as follows:

Assembly semantic vector: The task of SABLML in SABLML-VD is to generate semantic-aware assembly code feature vectors. For the input token sequence $\mathbb{T}_{asm}$ of the assembly code function $\mathbb{F}_{asm}$, the SABLML transformer encoder outputs sequences of hidden states. According to existing research [11,15], we take the hidden states of the last layer as the assembly semantic vector, then we generate the assembly code feature vector through a mean pooling layer, denoted as V_{asm} .

Pseudocode embedding: For the assembly code function $\mathbb{F}_{asm}$, we use the high-performance decompilation module of IDA [18] to generate the pseudocode function, denoted as $\mathbb{F}_{psc}$. Since the pseudocode has the same syntactic structure as normal source code, it can be applied to the source code language model. We utilize the effective and well-performing source code language model UniXcoder [33] to tokenize $\mathbb{F}_{psc}$ and generate the pseudocode embedding. Subsequently, we extract the dimension corresponding to the `<cls>` token as the pseudocode feature vector, denoted as V_{psc} .

The weights of SABLML and UniXcoder are also part of the training process for SABLML-VD (this can be regarded as a fine-tuning process independent of the pre-training stage). They are combined with the subsequent vulnerability classification part, and gradient descent is implemented by calculating the vulnerability binary classification loss in Section 3.1 to optimize the weights.

3.4.2. Vulnerability classification

We explored the effective fusion methods of assembly code feature vectors and pseudocode feature vectors to achieve the optimal vulnerability detection performance. The experimental content can be seen in Section 5.2(a). The experimental results indicated that concatenation had the relatively best performance. Therefore, we fuse the assembly code feature vector V_{asm} and the pseudocode feature vector V_{psc} by concatenation as the input V for the classifier. Subsequently, we employ MLP in conjunction with ResNet connections to construct a vulnerability classifier. The ResNet connections serve to enhance the model's capability in learning effective features and expedite model convergence [40].

$$V = \text{Concat}(V_{asm}, V_{psc}) \quad (8)$$

MLP comprises 2 layers of feed-forward neural networks, featuring geometrically decreasing dimensions. These layers are interconnected using the activation function $\text{Tanh}(\cdot)$, batch normalization, and residual connection. For the l th layer, we design it as follows:

$$FFN_l : \mathbb{R}^{d_v/2^{(l-1)}} \rightarrow \mathbb{R}^{d_v/2^{(l)}} \quad (9)$$

$$V^{(l)'} = \text{Tanh}(\text{BatchNorm}(FFN_l(V^{(l)}))) \quad (10)$$

$$V^{(l+1)} = V^{(l)'} \oplus V^{(l)} \quad (11)$$

where d_v is the dimension of the comprehensive feature vector V , $V^{(l)}$ is the input of V at the l th layer.

Finally, we add the 3rd layer, a feed-forward neural network with an output dimension of 2. The function $\text{Softmax}(\cdot)$ is applied to this layer to perform vulnerability detection, giving the final binary classification result $\hat{y}$.

$$\hat{y} = \text{Softmax}(FFN(V^{(3)})) \quad (12)$$

4. Experimental setup

4.1. Research questions

To evaluate and validate the performance and effectiveness of our proposed method in the task of vulnerability detection, we pose the following research questions, among which RQ1, RQ2, RQ5, RQ6, and RQ7 are the evaluations of SABLM-VD, and RQ3 and RQ4 are respectively the evaluations of code tokenization and pre-training strategies in SABLM:

RQ1: How effective is SABLM-VD compared to state-of-the-art vulnerability detection baselines?

RQ2: What is the impact of different components on the detection performance of SABLM-VD?

RQ3: How effective is hybrid-granularity code tokenization?

RQ4: How effective is the pre-training strategy of SABLM?

RQ5: How robust is SABLM-VD?

RQ6: How effective is SABLM-VD in real-world applications?

RQ7: How efficient is SABLM-VD in detecting vulnerabilities?

4.2. Experiment implementation

In terms of hardware, we utilize a server equipped with an NVIDIA GeForce RTX 4090 GPU, an Intel Xeon Platinum 8352V CPU, and 90 GB of RAM. In terms of software, we utilize IDA Pro 7.0 to disassemble and decompile binary files. We primarily employ Python 3.8, PyTorch 1.12.0, Ubuntu 20.04, and CUDA 11.3 to implement our models, SABLM and SABLM-VD.

We employ UniXcoder [33] for pseudocode embedding. We set the attention head parameters $\theta_1, \theta_2, \theta_3, \theta_4$ in SABLM pre-training and the attention head parameters in SABLM-VD training to 12. For DTP, ALP, and CFP, the masking ratio parameters θ_{dtp} , θ_{alp} , and θ_{cfp} are all set

Table 5

NVD dataset. “Arch”, “Com”, “Ver”, and “Optimization” represent architecture, compiler, compiler version, and optimization level, respectively. The numerical format indicates (No-vul:Vul).

Arch	Com	Ver	Optimization			
			O0	O1	O2	O3
x86-64	GCC	4.8.0	314:440	243:447	249:448	233:417
		7.5.0	314:413	251:475	250:455	235:409
	Clang	6.0	283:361	270:341	194:239	191:238
		8.0	283:361	271:343	196:242	193:240
ARM	GCC	4.8.0	135:175	101:151	95:141	85:126
		7.5.0	134:180	101:180	101:173	92:158
Total			4,814 : 7,153			

to 15% to maintain compatibility with the original design of BERT. The batch sizes for training SABLM and SABLM-VD are set to 16 and 8, respectively, with both models undergoing 8 epochs and having a learning rate of $2e^{-5}$. During the MLM pre-training process, a warmup mechanism [40] is applied, whereas during the DTP+ALP+CFP pre-training process, the warmup mechanism is disabled. For both SABLM and SABLM-VD, AdamW [41] is employed, and the optimal model is selected based on the highest F1-score achieved on the test set. To parallelize the calculation, the samples at the end of the dataset that do not meet the batch size are set to be dropped.

4.3. Dataset

(1) NVD dataset: There is not yet an open-source, function-level binary code NVD dataset. Therefore, we collected a large amount of vulnerability/patch source code from the NVD and compiled it into binary code to form the NVD dataset.

Specifically, we collected function-level vulnerability/patch source code for 450 CVEs [17] from 8 representative open-source projects (Linux, OpenSSL, Qemu, FFmpeg, Libav, Xen, Tcpdump, Libtiff). Since individual functions are not directly compilable, we built complete compilation environments with libraries for both x86-64/ARM architectures and GCC/Clang compilers, based on CVE-described project versions. These source codes were compiled under 4 optimization levels (O0–O3) to obtain vulnerability/patch object *.o* binary files. As *.o* files strictly correspond to *.c* files and can be linked into executables *.elf* files, they serve as ideal binary vulnerability samples. The entire compilation proved challenging: Projects with complex dependencies (e.g., Linux 4.9.1 contains 28,018 *.c* and 24,713 *.h* files) required full-project compilation per CVE version to avoid omissions/errors/failures, significantly increasing workload. Some CVE versions also lacked support for specific architectures/compilers, causing sample gaps. Finally, we disassembled binaries via IDA to extract CVE-related assembly functions, forming the NVD dataset.

In the NVD dataset, we considered the CVE vulnerability functions as binary vulnerability samples (denoted as “Vul”), totaling 7,153, and the CVE patch functions as binary benign samples (denoted as “No-vul”), totaling 4,814. Detailed information is presented in Table 5. We randomly split the vulnerability/benign samples under each compilation configuration into training, validation, and test sets at an 8:1:1 ratio, forming the NVD (Mixed) dataset. Additionally, to evaluate cross-architecture performance, we adjust the sample architectures of the training set to be different from those of the validation and test sets, forming two cross-architecture datasets: the “x86-64->ARM” dataset (i.e., the training set contains only x86-64 samples, while the validation/test sets contain only ARM samples) and the “ARM->x86-64” dataset (i.e., the training set contains only ARM samples, while the validation/test sets contain only x86-64 samples). Then, we randomly sample to ensure that the sample ratios of the training, validation, and

Table 6
SARD dataset.

Architecture	No-vul	Vul	Total
x86-64	40,664	40,746	81,410
ARM	41,423	41,456	82,879

test sets in both cross-architecture datasets are 8:1:1, and these two datasets together form the NVD (XA) dataset.

(2) SARD dataset: We collected C/C++ source codes from SARD [16] and compiled them to form the SARD dataset. Through expert experience, the SARD organization has artificially constructed source code test suites with vulnerability characteristics to evaluate the performance of static vulnerability detection tools. The C/C++ test suites, which have been used as evaluation datasets by multiple methods [13, 14, 23], contain a large number of generated, synthetic, and academically researched source code programs. These programs are categorized based on different CWEs (Common Weakness Enumeration). Under each CWE, they are further divided into “bad” (vulnerable) and “good” (patched).

We built a GCC compilation environment under ARM/x86-64 architecture, adopted the default optimization level, and compiled all SARD C/C++ source code programs into binary files for both ARM and x86-64 architectures, covering a total of 83 CWE categories. Through IDA disassembly, we extracted “bad”/“good” assembly code functions from the binary files to form the SARD dataset.

In the SARD dataset, we classify “bad” functions as binary vulnerability samples (denoted as “Vul”), and “good” functions as binary benign samples (denoted as “No-vul”). Specific information is shown in Table 6. We randomly split the x86-64 and ARM architecture vulnerability/benign samples in an 8:1:1 ratio into training sets, validation sets, and test sets, respectively, obtaining the SARD (x86-64) and SARD (ARM) datasets. Then, considering that the sample size difference between the SARD (x86-64) and SARD (ARM) datasets does not exceed 2%, we obtained two cross-architecture datasets “x86-64->ARM” and “ARM->x86-64” by directly swapping the training sets of SARD (x86-64) and SARD (ARM), and these two datasets together form the SARD (XA) dataset. In addition, unlike the NVD dataset, the SARD dataset may contain a small amount of residual source code information generated by IDA that could affect the judgment of the detection model (such as CWE-121, good, bad, etc.). To address this, we uniformly replaced relevant keywords in all types of SARD datasets. Specifically, we replaced Good/good/Bad/bad with X, and function names containing CWE were replaced with FUNCX (e.g., `CWE121_Stack_Based_Buffer_Overflow__char__type_overrun_memcpy_13_bad` will be replaced with `FUNCX`).

(3) Assembly corpus: To effectively pre-train SABLM, we constructed a sizable assembly code corpus reflecting real-world contexts. Specifically, we extracted function-level assembly code unrelated to vulnerabilities/patches from binaries of 8 open-source projects (Linux, OpenSSL, Qemu, FFmpeg, Libav, Xen, Tcpdump, and Libtiff). These binary files, which are intermediary files obtained during the acquisition of the NVD dataset, were compiled across compilers, optimization levels, and x86-64/ARM architectures. As a single .o file contains hundreds to thousands of functions, we used IDA disassembly to extract all functions for the corpus. We then removed vulnerability/patch-related functions from the NVD dataset to avoid impacting detection model evaluation, retaining only irrelevant functions for the assembly language model.

Ultimately, the assembly corpus comprises 336,939 x86-64 assembly function corpora and 57,935 ARM assembly function corpora. We randomly mixed the corpora from both architectures and divided them into non-overlapping training, validation, and test sets in an 8:1:1 ratio. During the two-step pre-training process, we selected the optimal model based on the lowest $\mathcal{L}_{SABLM}^{(1)}$ and $\mathcal{L}_{SABLM}^{(2)}$ on the assembly corpus test set, respectively.

Table 7

Real-world codes. “Vul” represents the number of vulnerable functions, and “Irr” represents the number of irrelevant functions.

Project	CVE	Vul	Irr
Linux	CVE-2013-1792,CVE-2014-9529, CVE-2018-14633,CVE-2019-14821, CVE-2020-8648	6	16
OpenSSL	CVE-2015-0286,CVE-2016-0797, CVE-2016-2180,CVE-2016-6303, CVE-2019-1543	6	21
Qemu	CVE-2011-1751,CVE-2013-4539, CVE-2014-0147,CVE-2014-3471, CVE-2016-4441	12	95
FFmpeg	CVE-2013-3672,CVE-2013-3674, CVE-2014-8544,CVE-2017-9992, CVE-2020-22019	5	58
Libav	CVE-2011-3940,CVE-2011-3952, CVE-2012-0947,CVE-2012-2777, CVE-2012-2793	5	35
Xen	CVE-2014-4021,CVE-2015-7969, CVE-2016-9378,CVE-2020-29567, CVE-2021-28701	6	107
Tcpdump	CVE-2016-7925,CVE-2016-7926, CVE-2016-7928,CVE-2017-5484, CVE-2017-13687	5	7
Libtiff	CVE-2008-2327,CVE-2013-4231, CVE-2014-8128,CVE-2016-3623, CVE-2018-8905	6	34

(4) Real-world codes: The real-world binary code is typically extracted from firmware and often contains not only potential vulnerability/patch functions but also a large number of irrelevant functions, which is different from the NVD dataset. To evaluate the detection capability of our model on real-world code, we randomly selected binary files corresponding to 5 distinct CVEs under different compilation configurations from the procedural .o binary files of 8 open-source projects in the test and validation sets of the NVD dataset, totaling 44 CVE-related binary files (some CVEs involve multiple binaries). From these files, we extracted all vulnerability functions (positive samples) and irrelevant functions (negative samples) to form a real-world code test set. Details are presented in Table 7. In addition, we have also collected four kinds of open-source projects whose versions are outside the NVD dataset to discover potential unknown vulnerabilities. These open-source projects include: FFmpeg 0.8.6/2.2.1, Libav 0.8, Qemu 2.3.0/3.1.0, and Xen 4.15.1.

4.4. Baseline methods

We consider comparing SABLM-VD with state-of-the-art graph-based methods [42–44], sequence-based methods [10,11,21,45,46], and large language models (LLM) [47,48]. We try our best to implement the baselines based on the original papers and their corresponding environments and parameter settings. Meanwhile, we retrain/fine-tune all baselines except LLMs and CLAP on the NVD and SARD datasets in this paper. For LLMs, to maintain their original weights and performance, they are not involved in the training/fine-tuning on the training set of this paper but are directly used for performance testing. The test example of the baseline LLM is shown in Fig. 6.

(1) GGSNN: GGSNN [42] is a GNN model with gating units, proposed by Li et al. In this paper, we use GGSNN to learn the CFG extracted from assembly code for vulnerability detection.

(2) GMN: GMN [43] determines similarity by calculating the cross-graph attention of the CFG. In this paper, we add fully connected layers and a classifier after generating the graph feature vectors in GMN to identify vulnerable functions.

(3) GNN-s2v: GNN-s2v [44] is a GNN model that uses Structure2vec (s2v) [49] for CFG node embedding. In this paper, we select

Table 8

General performance comparison results of SABLM-VD with baselines on three datasets (note that the 2 implementations of GNN-s2v are not applicable to the NVD (Mixed) dataset due to underfitting). The best result for each metric is highlighted in bold. The shaded cells represent the performance of the Top-3 best methods in each metric. Darker cells represent better performance.

Metric (%) \ Dataset	NVD (Mixed)				SARD (x86-64)				SARD (ARM)			
	F1	Pre	Rec	Acc	F1	Pre	Rec	Acc	F1	Pre	Rec	Acc
Baseline												
GGSNN	6.58	57.50	3.49	41.70	40.02	27.54	73.21	33.46	40.49	26.71	83.63	27.86
GMN	67.76	57.99	81.49	54.37	45.14	29.65	94.54	30.36	46.04	30.10	97.85	32.61
GNN-s2v _(att)	–	–	–	–	69.44	71.07	67.88	81.52	65.36	63.01	67.89	80.22
GNN-s2v _(lstm)	–	–	–	–	79.85	79.88	79.83	87.54	68.11	64.43	72.24	81.41
Llama-3	70.11	59.83	84.65	56.82	62.80	49.15	86.95	48.53	59.72	47.19	81.33	44.67
DeepSeek-R1	69.30	60.52	81.05	56.13	69.63	61.38	80.44	63.76	65.20	56.82	76.47	57.63
CLAP	48.94	62.47	40.22	49.79	37.10	40.12	34.50	41.50	33.92	76.99	21.75	57.63
SAFE	71.80	59.33	90.92	57.86	71.77	78.54	66.07	84.23	66.39	65.80	66.99	81.70
PalmTree	71.15	61.53	84.34	59.15	94.25	94.33	94.16	94.19	76.21	65.21	91.66	71.65
jTrans	68.23	68.79	67.69	62.96	98.69	98.39	98.99	98.70	97.42	96.84	98.00	97.40
HAformer	72.54	61.48	88.40	61.23	99.01	99.50	98.52	99.01	98.64	97.81	99.48	98.62
SABLM-VD	79.33*	69.39*	92.59	71.18*	100.00*	100.00*	100.00*	100.00*	100.00*	100.00*	100.00*	100.00*

* Denotes statistical significance in comparison to HAformer and the metric is higher than HAformer (i.e., two-sided t-test with p -value < 0.05).

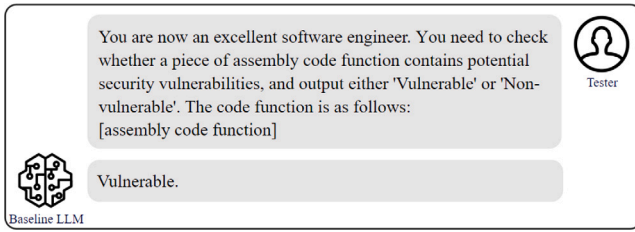


Fig. 6. Example of baseline LLM test for a single sample.

two excellent variants from its multiple variants for experimentation: GNN-s2v_(att), which generates embeddings by calculating node attention weights; and GNN-s2v_(lstm), which uses Long Short-Term Memory (LSTM) to learn node content sequences to generate embeddings.

(4) **SAFE**: SAFE [45] employs the Word2vec model to generate assembly sequence embeddings, followed by a self-attentive bidirectional RNN to generate function embeddings.

(5) **PalmTree**: PalmTree [10] learns the control and data relationships of assembly code through the CWP and DUP pre-training tasks. We fine-tune the original weights of PalmTree for vulnerability detection in this paper.

(6) **jTrans**: jTrans [11] is a BERT-based binary language model that pre-trains on assembly code jump instructions to understand the control flow characteristics of assembly code. In this paper, we add fully connected layers and a classifier to the original jTrans model and fine-tune the overall weights for vulnerability detection.

(7) **HAformer**: HAformer [21] combines assembly code with hexadecimal machine code, utilizing a Transformer-based model to detect vulnerabilities.

(8) **CLAP**: CLAP [46] is a language model applicable to various zero-shot downstream tasks in binary analysis. Considering its zero-shot characteristic, this paper directly invokes the CLAP prototype for testing. Specifically, we input assembly code along with two types of prompts, “This is a piece of vulnerable code” and “This is a piece of non-vulnerable code”, and then perform binary classification based on the output probabilities of CLAP.

(9) **Llama-3**: Llama-3 [47], proposed by Meta, employs Grouped Query Attention (GQA) to enhance long context understanding as an LLM.

(10) **DeepSeek-R1**: DeepSeek-R1 [48], proposed by DeepSeek, is a high-performance LLM that employs unsupervised reinforcement learning and Group Relative Policy Optimization (GRPO).

4.5. Performance metrics

We use the following 4 widely utilized evaluation metrics to measure the performance of the model. In the metrics, TP , TN , FP , FN denote the number of true positives, true negatives, false positives, false negatives, respectively.

Precision: Precision (Pre) is the percentage of true vulnerability samples among the detected vulnerability samples.

$$Pre = \frac{TP}{TP + FP} \quad (13)$$

Recall: Recall (Rec) is the percentage of the detected vulnerability samples among all vulnerability samples.

$$Rec = \frac{TP}{TP + FN} \quad (14)$$

F1-score: F1-score is the harmonic mean between Pre and Rec , reflecting the overall performance of the model to detect vulnerabilities.

$$F1\text{-score} = 2 \times \frac{Pre \times Rec}{Pre + Rec} \quad (15)$$

Accuracy: Accuracy (Acc) is the percentage of samples that are correctly classified over all samples.

$$Acc = \frac{TP + TN}{TP + TN + FP + FN} \quad (16)$$

5. Experimental result

5.1. Overall performance analysis (RQ1)

To answer RQ1, we first analyze the performance comparison between SABLM-VD and baselines. Next, we explore cross-architecture comparisons. Finally, we verify the learned vulnerability patterns via feature visualization.

(a) **General performance comparison**: Table 8 presents the overall results of SABLM-VD and the baselines on the four metrics of the NVD (Mixed), SARD (x86-64), and SARD (ARM) datasets (* indicates the significance test results compared with the closest-performing HAformer). SABLM-VD outperforms the other baseline methods, with F1-scores that are 6.79%, 0.99%, and 1.36% higher on the three datasets, respectively. Moreover, SABLM-VD achieves the best performance in all 12 comparisons (metrics $\times$ datasets).

SABLM-VD achieves 100.00% on all 4 metrics for both SARD (x86-64) and SARD (ARM) datasets, demonstrating perfect identification of vulnerable and benign functions from these two datasets (note: impact of special SARD keywords like CWE121/good/bad removed; see Section.4.3). On NVD (Mixed), SABLM-VD significantly outperforms all

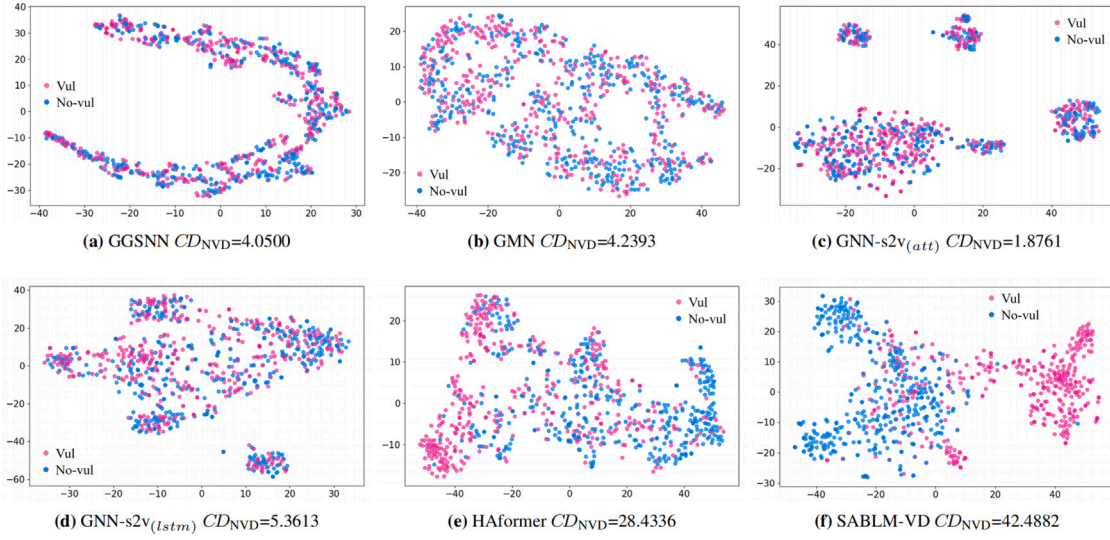


Fig. 7. The t-SNE visualizes NVD (Mixed) test set distributions: vulnerable (pink) vs. non-vulnerable (blue) samples across methods. CD_{NVD} denotes centroid distance.

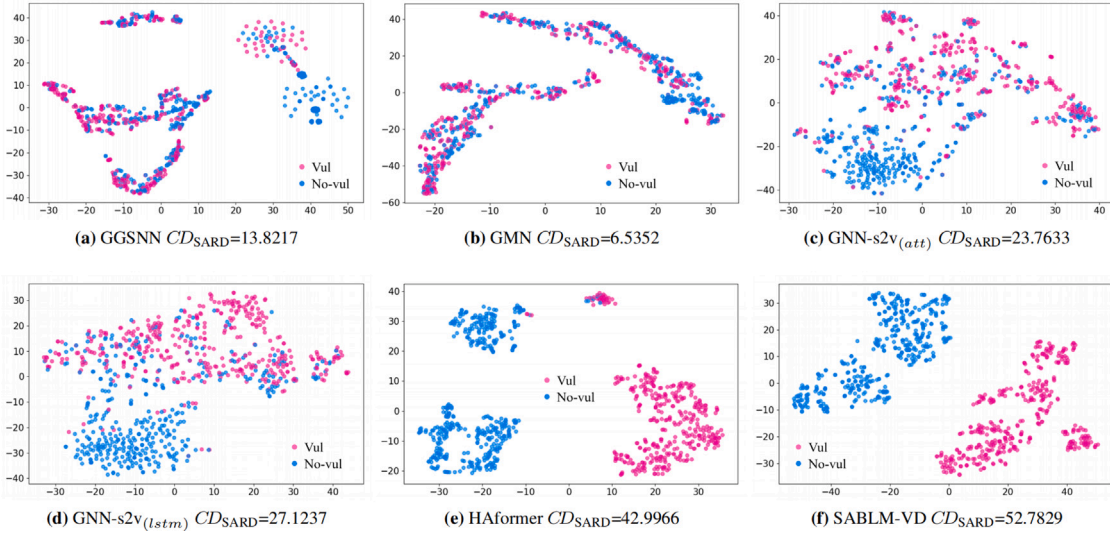


Fig. 8. The t-SNE visualizes SARD (x86-64) test set distributions: vulnerable (pink) vs. non-vulnerable (blue) samples across methods. CD_{SARD} definition same as CD_{NVD} .

Table 9

Cross-architecture performance comparison results of SABLM-VD with baselines on two datasets.

Metric (%)	F1-score			
	NVD (XA)		SARD (XA)	
Dataset				
Type	x86-64->ARM	ARM->x86-64	x86-64->ARM	ARM->x86-64
Train	x86-64	ARM	x86-64	ARM
Test	ARM	x86-64	ARM	x86-64
GGSNN	68.37	63.37	45.29	43.73
GMN	69.85	62.76	45.42	45.73
GNN-s2v _(att)	—	—	19.06	23.73
GNN-s2v _(lstm)	—	—	24.63	35.76
SAFE	36.41	28.32	22.86	30.80
jTrans	70.42	72.70	66.67	66.04
HAformer	74.90	74.72	89.60	94.59
SABLM-VD	77.90	75.18	100.00	100.00

baselines in F1-score, precision, and accuracy. These gains are crucial for vulnerability detection: the higher F1-score indicates better overall detection performance, while higher precision and accuracy reflect a deeper understanding of vulnerability semantics.

Experimental results also show that attention-based methods (HAformer, jTrans) outperform non-attention methods (SAFE, GGSNN, GMN). For instance, on the NVD (Mixed) dataset, the shaded cells indicate that HAformer/jTrans achieved the Top-3 results five times, while SAFE/GGSNN/GMN only achieved it twice. The attention mechanism better captures vulnerability patterns, thus improving the association with code features and semantic understanding. Furthermore, Table 8 shows LLM methods (Llama-3, DeepSeek-R1) did not achieve Top-3 rankings across all metrics on the 3 datasets. This is likely due to their lack of prior binary vulnerability knowledge, causing detection errors.

(b) Cross-architecture performance comparison: Table 9 presents the F1-scores of SABLM-VD and 7 baselines on NVD (XA) and SARD (XA) datasets (LLM methods excluded as untrained). SABLM-VD achieves the best cross-architecture performance, outperforming other baselines by 3.00%, 0.42%, 10.40%, and 5.41% F1-score in the 4 training+testing scenarios, respectively.

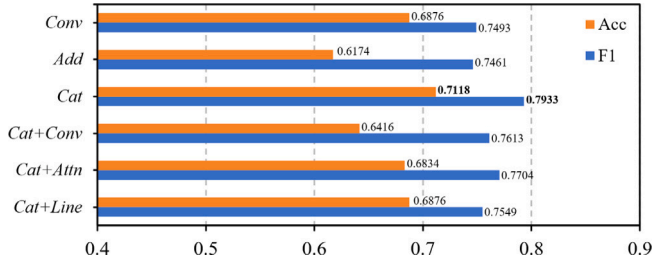


Fig. 9. The performance test results of different feature fusion methods on the NVD (Mixed) dataset.

Additionally, when trained on SARD (XA), 5 of the 7 baselines show generally lower F1-scores compared to single-architecture results in Table 8. However, GGSNN and GMN exhibit slight improvements. We believe binary code's structural features help mitigate architectural differences.

(c) Feature visualization: To analyze the effectiveness of SABLM-VD in vulnerability detection, we employ t-SNE [50] to visualize the learned features of binary code and compare them with the baselines, including GGSNN, GMN, GNN-s2v_(att), GNN-s2v_(lstm), and HAformer. In this t-SNE space, a greater distance between “Vul” (positive) and “No-vul” (negative) nodes indicates clearer class separability and higher model performance. We also quantify this separability using Centroid Distance CD [51].

For intuitive visualization, we randomly selected equal positive and negative samples from the NVD (Mixed) and SARD (x86-64) test sets for t-SNE. Figs. 7 and 8 show the t-SNE visualizations for GGSNN, GMN, GNN-s2v_(att), GNN-s2v_(lstm), HAformer, and SABLM-VD, with CD_{NVD} and CD_{SARD} representing the centroid distances on each dataset.

On the NVD (Mixed) dataset, GGSNN, GMN, GNN-s2v_(att), and GNN-s2v_(lstm) show severely mixed positive/negative samples, indicating poor separability (all $CD_{NVD} < 6$ in Fig. 7(a-d)). The mixing also reflects the underfitting of GNN-s2v_(att) and GNN-s2v_(lstm). HAformer shows preliminary bipolarization (Fig. 7(e)). SABLM-VD achieves the highest CD_{NVD} (42.4882), with multipolarization, clear boundaries, and obvious separability (Fig. 7(f)).

On the SARD (x86-64) dataset, GGSNN and GMN samples remain highly mixed (Fig. 8(a, b)). GNN-s2v_(att) and GNN-s2v_(lstm) show vague boundaries (Fig. 8(c, d)). HAformer still has few samples in incorrect regions (Fig. 8(e)). SABLM-VD achieves the highest CD_{SARD} (52.7829), with all samples in two distant regions, demonstrating significant separability (Fig. 8(f)). This is consistent with the 100.00% F1-score of SABLM-VD on this dataset.

The above visualization results further demonstrate the effectiveness of SABLM-VD in distinguishing vulnerable code from non-vulnerable code.

Answer to RQ1: Whether in general performance comparison or cross-architecture performance comparison, SABLM-VD outperforms all baselines in terms of F1-score. Visualization further demonstrates that SABLM-VD can better distinguish code vulnerabilities than the baselines.

5.2. Component analysis (RQ2)

To answer RQ2, we first explore feature fusion methods between SABLM and pseudocode on NVD (Mixed). Then, using the best fusion, we perform an ablation study on SABLM-VD to evaluate the impact of DTP, ALP, CFP in SABLM and the pseudocode part.

(a) Feature fusion analysis: For feature vectors from SABLM and pseudocode part, we tested 6 common fusion methods: convolution

Table 10

The ablation study results of SABLM-VD on the NVD (Mixed) dataset.

Metric (%) \ Dataset	NVD (Mixed)			
	F1	Pre	Rec	Acc
- DTP (SABLM)	75.21	69.39	82.10	67.67
- ALP (SABLM)	75.49	68.49	84.08	67.34
- CFP (SABLM)	77.38	69.32	87.55	69.42
- SABLM	72.34	63.25	84.48	61.40
- Pseudocode	77.58	66.97	92.17	68.17
SABLM-VD	79.33	69.39	92.59	71.18

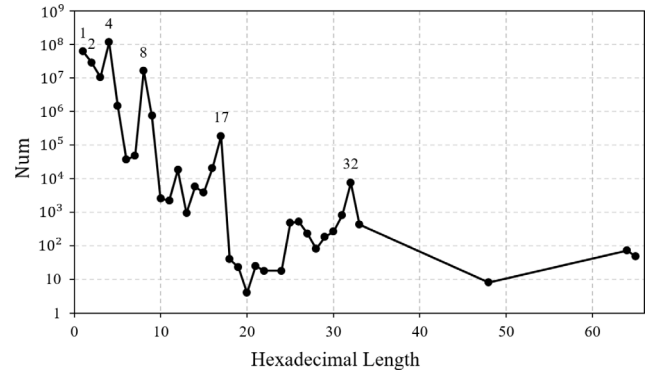


Fig. 10. The count of hexadecimal numbers of different lengths.

(Conv), addition (Add), concatenation (Cat), concatenation+convolution (Cat+Conv), concatenation+attention-layer (Cat+Attn), and concatenation+ linear-layer (Cat+Line). Fig. 9 shows F1-score and accuracy results. Concatenation achieves optimal metrics. We attribute this to the inherent semantic richness of the BERT output vectors, which can degrade semantics through complex fusion.

(b) Ablation study: Based on the concatenation fusion, we constructed 5 SABLM-VD variants: (1) -DTP (SABLM): Remove DTP pre-training to verify data transfer semantic awareness impact; (2) -ALP (SABLM): Remove ALP pre-training to verify arithmetic logic semantic awareness impact; (3) -CFP (SABLM): Remove CFP pre-training to verify control flow semantic awareness impact; (4) -SABLM: Remove entire SABLM to assess its overall contribution; (5) -Pseudocode: Remove pseudocode embedding to evaluate pseudocode feature impact.

Ablation results (Table 10) show all variants underperform SABLM-VD, confirming that the pseudocode embedding and the DTP, ALP, and CFP tasks of SABLM all contribute to performance. Specifically, removing any one of the pre-training tasks leads to an average decrease of 3.30%, 0.32%, 8.01%, and 3.04% in F1 score, precision, recall, and accuracy, respectively. Removing SABLM causes these metrics to drop by 6.99%, 6.14%, 8.11%, and 9.78%. Removing the pseudocode results in a decrease of 1.75%, 2.42%, 0.42%, and 3.01% in these metrics.

The above results confirm that SABLM contributes more substantially than pseudocode embedding, with DTP being the most impactful component in SABLM. This likely stems from data/arithmetic/control pre-training, which enhances holistic code semantics understanding, where data transfer semantics show a higher vulnerability pattern relevance. Notably, while individual DTP/ALP/CFP minimally affect precision/accuracy, their combination (SABLM) significantly boosts these metrics, indicating synergistic effects for precise vulnerability-benign function discrimination.

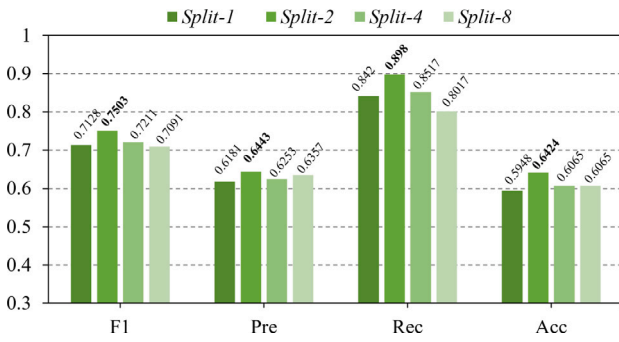


Fig. 11. Performance comparison of variants with different splitting intervals on the NVD (Mixed) dataset.

Table 11

Comparison of vocabulary sizes of different variants with different splitting intervals.

Variant	Split-1	Split-2	Split-4	Split-8
Vocabulary Size	39,631	39,887	50,255 ^a	50,255 ^a

^a Indicates that the upper limit allowed by the BERT-based model has been reached.

Table 12

Performance comparison results of different granularity tokenization methods on the NVD (Mixed) dataset.

Metric (%) \ Dataset	NVD (Mixed)			
	F1	Pre	Rec	Acc
Granularity				
Subword-1	74.68	66.48	85.17	65.50
Subword-2	75.17	68.92	82.68	67.34
Word	73.31	69.79	77.20	66.42
Hybrid-granularity	79.33	69.39	92.59	71.18

Answer to RQ2: Concatenation is an effective way to fuse the features of SABLM and pseudocode. The DTP, ALP, and CFP pre-training tasks in SABLM, and the pseudocode embedding components can effectively improve the performance of SABLM-VD. SABLM contributes the most to the overall performance.

5.3. Effectiveness of hybrid-granularity code tokenization (RQ3)

To answer RQ3, we first analyze the effective split units for hexadecimal numbers at hybrid granularity. Then, we explore the performance comparison between SABLM-VD using hybrid-granularity code tokenization and its variants that employ other tokenization methods.

(a) Hexadecimal number splitting: Hexadecimal numbers, the most diverse assembly elements, require optimal splitting intervals (overly large intervals expand vocabulary, while small ones lengthen token sequences). We determine optimal intervals by scaling frequent hexadecimal lengths to powers of two. These numbers appear as immediates, addresses, offsets, and constants with varying lengths. Corpus statistics (Fig. 10) show the Top-4 lengths (4,1,2,8) all exceed 10^7 , are all powers of 2, and account for 94.52% of the total. Thus, we use 1, 2, 4, and 8 as split intervals. It should be noted that statistics are derived from IDA-disassembled x86-64/ARM disassembly code, which differs from forward-developed assembly code but aligns with the needs of binary analysis.

We apply splitting intervals 1/2/4/8 to SABLM-VD variants for performance comparison. Specifically, four pseudocode-excluded variants are trained/tested on NVD (Mixed) with these intervals (to eliminate

the influence of pseudocode), denoted as *Split-1*, *Split-2*, *Split-4*, and *Split-8*.

Fig. 11 results show *Split-2* outperforms others in F1-score, precision, recall, and accuracy, confirming 2 as the optimal hexadecimal splitting interval. Furthermore, in the vocabulary statistics (Table 11), the *Split-4*/*Split-8* models reach the vocabulary limit, while *Split-1*/*Split-2* are below the limit. Moreover, the vocabulary of *Split-2* is only 0.65% smaller than that of *Split-1*, indicating that an interval of 2 can effectively alleviate the OOV problem.

(b) Hybrid-granularity code tokenization: Based on the most effective hexadecimal splitting, we propose hybrid-granularity code tokenization. To evaluate it, we build SABLM-VD variants with different tokenization for performance comparison. However, as SABLM in SABLM-VD is designed, trained, and implemented on hybrid-granularity tokenization, it cannot directly use other methods. Thus, for variants with other tokenizations, we employ corresponding language models to generate assembly code embeddings from token sequences. We create two subword-level variants: embeddings by CodeBERT and UniXcoder, labeled *Subword-1* and *Subword-2*, and one word-level variant with jTrans embeddings, labeled *Word*.

Performance comparisons are in Table 12. The hybrid-granularity outperforms other variants, verifying its effectiveness. Notably, the *Word* variant reduces F1-score by 6.02%, recall by 15.39%, and accuracy by 4.76%, but improves precision by 0.40%. This may occur because word-level granularity preserves longer original sequences with fewer tokens, despite lacking operand subdivision, which affects detection performance.

Answer to RQ3: 2 is the most effective splitting interval for hexadecimal numbers and can alleviate the OOV problem. The hybrid-granularity code tokenization based on interval 2 is effective.

5.4. Effectiveness of pre-training strategy of SABLM (RQ4)

The research results of the ablation study on RQ2 clearly indicate that the absence of DTP, ALP, or CFP in SABLM results in a significant decrease in SABLM-VD's performance, highlighting the effectiveness of DTP, ALP, and CFP. However, whether these SABLM pre-training strategies are sufficiently effective remains a question that we need to answer. To this end, we first explore the impact of different types of opcode masking strategies within DTP, ALP, and CFP on the performance of SABLM-VD. Then, we also explore the influence of other potentially effective pre-training tasks on the performance of SABLM-VD.

(a) DTP, ALP, and CFP masking: Different types of masking for DTP, ALP, and CFP are shown in Table 2. We constructed 13 SABLM-VD variants by unmasking a single masking type (e.g., DTP-1, DTP-2, ALP-2, etc.) or a single masking position within a masking type (e.g., <dtp_2a>, <dtp_2b>, etc.) to analyze the contributions of different maskings.

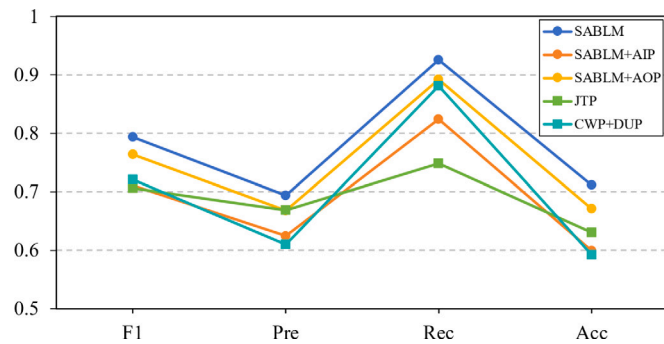
Results in Table 13 show all variants underperforming original SABLM-VD in F1-score, precision, and accuracy, confirming that the DTP, ALP, and CFP mask types all contribute to the performance. Notably, DTP-1 masking contributes the most to recall (6.42%), as pre-training on push/pop instructions enhances the model's understanding of stack-related vulnerabilities. <dtp_2a> contributes the most to precision and accuracy (3.26% and 4.26%), as data-transfer instructions (e.g., mov) determine data-flow direction, influencing local semantics. ALP-3 masking contributes the most to F1-score (2.83%), because three-operand arithmetic logic instructions critically affect computations.

(b) Other pre-training strategies: For the SABLM language model, in addition to the basic MLM task, the DTP/ALP/CFP pre-training strategies are specifically designed for assembly instruction operands

Table 13

Performance comparison of variants with different masking types on the NVD (Mixed) dataset. “Pre-training” column indicates the pre-training objective to which the masking type belongs. ↓ indicates the variant with the largest decrease in the metric of that column.

Pre-training	Type	F1	Pre	Rec	Acc
DataTransfer	- DTP-1	-2.78	-0.53	↓-6.42	-2.76
	- DTP-2	-2.77	-2.44	-3.20	-3.93
	- <dtb_2a>	-2.57	↓-3.26	-1.12	↓-4.26
	- <dtb_2b>	-0.70	-1.96	+1.68	-1.84
ArithmeticLogic	- ALP-1	-1.14	-2.02	+0.57	-2.26
	- ALP-2	-2.13	-1.11	-3.78	-2.51
	- <alp_2a>	-2.00	-2.86	-0.28	-3.51
	- <alp_2b>	-2.13	-1.11	-3.78	-2.51
	- ALP-3	↓-2.83	-1.38	-5.18	-3.26
	- <alp_3a>	-2.63	-0.54	-6.02	-2.59
	- <alp_3b>	-2.57	-0.97	-5.16	-2.84
	- <alp_3c>	-1.72	-0.56	-3.64	-1.84
Control Flow	- CFP-1	-1.95	-0.07	-5.04	-1.76
SABLM-VD		79.33	69.39	92.59	71.18

**Fig. 12.** Performance comparison results of different pre-training strategies.

and do not involve opcodes. Thus, we design two opcode-focused strategies: (1) Associated Instruction Prediction (AIP): Mask DTP/ALP/CFP opcodes with tokens <aip_1>, <aip_2>, and <aip_3>, respectively, training models to recognize functional roles. (2) Adjacent Opcode Prediction (AOP): Mask two consecutive opcodes with <aop_1> and <aop_2>, enhancing contextual relationship awareness. We integrate AIP/AOP with existing DTP+ALP+CFP for SABLM pre-training, evaluating via SABLM-VD. For comparison, we also build SABLM-VD variants using JTP of jTrans [11] and CWP+DUP of PalmTree [10] (replacing only DTP+ALP+CFP, retaining other parts, including the pseudocode part).

The results on the NVD (Mixed) dataset are shown in Fig. 12. We find that SABLM+AIP and SABLM+AOP are lower than SABLM in all 4 metrics. This indicates that the AIP and AOP strategies have a negative impact on the performance of vulnerability detection. We believe that the reason may be that for neural network models with attention mechanisms, the types of opcodes are relatively few, and the additional opcode pre-training strategies may lead to over-understanding of the model. Meanwhile, the pre-training strategy of SABLM is also superior to JTP and CWP+DUP, with an average improvement of 9.28% over JTP and 8.02% over CWP+DUP in all 4 metrics.

Answer to RQ4: The pre-training strategy of SABLM is sufficiently effective and outperforms the existing pre-training strategies JTP and CWP+DUP. Additionally, the opcode pre-training strategies AIP and AOP have a negative impact on the detection performance.

Table 14

The F1-scores (%) of SABLM-VD and HAformer on the SARD (x86-64) test sets with different data insertion ratios.

Method	0%	5%	10%	20%	40%
HAformer	99.01	98.50	97.01	95.81	89.38
SABLM-VD	100.00	100.00	100.00	99.63	98.38

Table 15

The F1-scores (%) of SABLM-VD and HAformer on different code obfuscation type SARD (x86-64) test sets.

Method	None	FLA	SUB	BCF	Split	ALL
HAformer	99.01	93.35	98.25	94.19	95.52	78.90
SABLM-VD	100.00	99.88	100.00	99.62	100.00	94.14

5.5. Robustness (RQ5)

To answer RQ5, we evaluated the robustness of SABLM-VD through data insertion and code obfuscation experiments. We conducted experiments on SARD (x86-64) for SABLM-VD and HAformer (the one with the closest performance), and processed the SARD (x86-64) test set with data insertion and code obfuscation.

(a) Data insertion: In binary disassembly practice, inserting data into code is an abnormal operation that can interfere with the overall judgment of the detection model on the code. Therefore, we can evaluate the robustness of the model through data insertion. To obtain real-world data, we extracted all data lines starting with “.data”/“.rodata” from the original .asm assembly files in the assembly corpus, totaling 21.54 million lines. Then, we randomly selected data from these data lines and randomly inserted them between the lines of various types of code in each sample of the SARD (x86-64) test set (including assembly code, pseudocode, and machine code for HAformer). The proportion of the number of inserted data lines to the total number of lines after insertion was used as the insertion ratio. To evaluate the performance when dealing with different insertion ratios of data, we constructed SARD (x86-64) test sets with insertion ratios of 0%, 5%, 10%, 20%, and 40%.

The experimental results are shown in Table 14. We found that the F1-score of SABLM-VD exceeded that of HAformer under all data insertion ratios. Moreover, when the insertion ratio was 40%, the F1-score of SABLM-VD only decreased by 1.62% compared to when the insertion ratio was 0%, while the F1-score of HAformer dropped by 10.62%. These results indicate that SABLM-VD is robust in dealing with data insertion. We believe the reason is that the combination of hybrid-granularity code tokenization and pre-training strategy, which enhances SABLM-VD’s ability to recognize and understand data, enabling it to mitigate the impact of inserted irrelevant/interfering data significantly.

(b) Code obfuscation: As a technique against binary code analysis, code obfuscation can interfere with the local judgment of the detection model on the code. Therefore, we can use code obfuscation to evaluate the robustness of the model. Since code obfuscation is usually implemented in the compilation stage, we recompiled the source code corresponding to the SARD (x86-64) test set functions using Obfuscator-LLVM (O-LLVM) [52], and set the obfuscation type through compilation options during compilation. To evaluate the performance when dealing with various code obfuscations, we respectively constructed SARD (x86-64) test sets with the following obfuscation types: no obfuscation (denoted as *None*), Control flow flattening (denoted as *FLA*), Instruction substitution (denoted as *SUB*), Bogus control flow graph (denoted as *BCF*), Basic block splitting (denoted as *Split*), and the combination of the above 4 obfuscation types (i.e., *FLA+SUB+BCF+Split*, denoted as *ALL*).

The experimental results are shown in Table 15. In Table 15, the F1-score of SABLM-VD is higher than that of HAformer under each

Table 16

Details of 9 N-Day vulnerability instances detected by SABLM-VD, including CVE number, checked out function, function start address, binary file, file path, open-source project, and its version.

Project	Version	Binary file	File path	Checked out function	Start address	Confirmed N-Day CVE
FFmpeg	0.8.6	vp3.o	/libavcodec/	vp3_update_thread_context()	.text:0 × 4660	CVE-2011-3934 ^a
		smacker.o	/libavcodec/	smacker_decode_header_tree()	.text:0 × 22B3	CVE-2011-3944
	2.2.1	smacker.o	/libavcodec/	smka_decode_frame()	.text:0 × 1600	CVE-2015-8365
Libav	0.8	avs.o	/libavcodec/	avs_decode_frame()	.text:0 × 0000	CVE-2012-2801
Qemu	2.3.0	scsi-bus.o	/hw/scsi/	scsi_req_parse_cdb()	.text:0 × 0FA0	CVE-2015-5158 ^a
		vmware_vga.o	/hw/display/	vmsvga_fifo_run()	.text:0 × 01B0	CVE-2016-4453
	3.1.0	server.o	/nbd/	nbd_negotiate_send_rep_verr()	.text:0 × 05D0	CVE-2020-10761
		sd.o	/hw/sd/	sd_normal_command()	.text:0 × 1850	CVE-2020-13253
Xen	4.15.1	iommu.o	/xen/drivers/passthrough/vtd/	reassign_device_ownership()	.text:0 × 2D40	CVE-2021-28702 ^a

^a Indicates that the CVE number is completely absent from the NVD dataset.

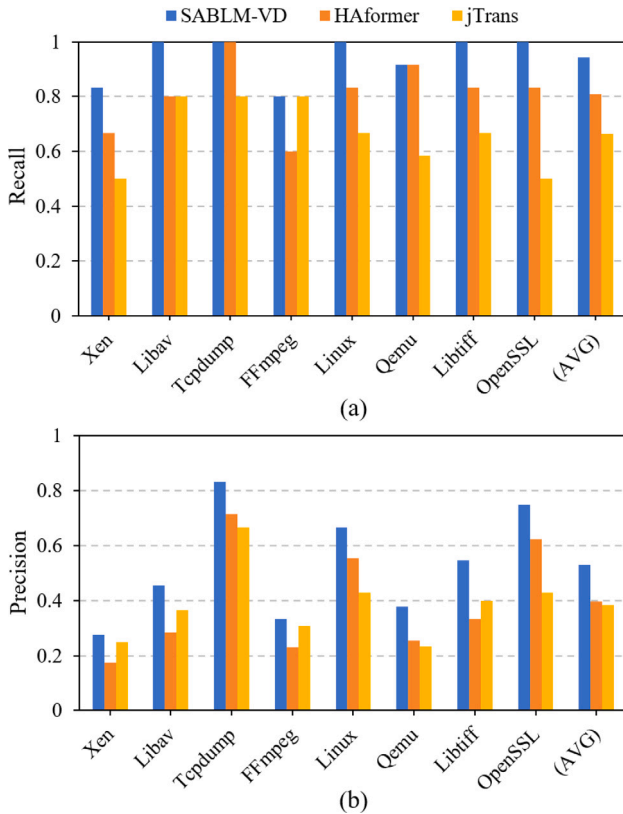


Fig. 13. The comparison results of recall and precision on 8 open-source projects in the real-world codes, with the rightmost column being the average value.

code obfuscation type. Meanwhile, compared with the no obfuscation case, the F1-score of SABLM-VD only decreases by 0.12% and 0.38% under *FLA* and *BCF* obfuscation, respectively, and by 5.86% under *ALL* obfuscation. However, the F1-score of HAformer drops by 20.11% under *ALL* obfuscation. These results indicate that SABLM-VD is robust in dealing with code obfuscation. We believe that the reason is that the semantic-aware pre-training strategy effectively enhances SABLM-VD's learning of semantics, alleviating the negative impact of code obfuscation on model performance.

Answer to RQ5: SABLM-VD is robust against data insertion and code obfuscation.

5.6. Real-world applications (RQ6)

To answer RQ6, first, we compare the performance of SABLM-VD and the baselines in detecting real-world vulnerabilities through the real-world code test set. The specific information of the real-world code test set is shown in Table 7. We trained SABLM-VD and the closest-performing HAformer, and jTrans on the NVD (mixed) dataset and then tested them on real-world codes. The results are shown in Fig. 13.

From the results, SABLM-VD achieves the highest recall and precision (including tied cases) in all 8 open-source projects. Moreover, its average recall is 13.33% and 27.92% higher than that of HAformer and jTrans, respectively, and its average precision is 13.33% and 14.52% higher than that of HAformer and jTrans, respectively. This indicates that SABLM-VD has the highest vulnerability detection capability in real-world codes. Notably, in terms of numerical values, SABLM-VD performs well in recall, but its precision exceeds 60% in only 3 projects. We believe the reason is that irrelevant functions, unlike high-security patched functions, may contain potential undiscovered 0-Day or N-Day vulnerabilities, leading to a small number of false positives in SABLM-VD. To this end, we manually analyzed all the irrelevant functions that were false positives in the real-world codes of Qemu and found that among the irrelevant functions in the same binary file *esp.o* as the CVE-2016-4441 vulnerability, there is an N-Day vulnerability function *esp_reg_write* highly related to CVE-2016-4439, which was not included in the NVD dataset of this paper.

Then, we used SABLM-VD alone to detect vulnerabilities in all binary functions of the four open-source projects (FFmpeg 0.8.6/2.2.1, Libav 0.8, Qemu 2.3.0/3.1.0, Xen 4.15.1) whose versions were not in the NVD dataset, and output the detection results. For those functions detected as having vulnerabilities, we conducted further manual analysis and verification to determine whether the function indeed had N-Day vulnerabilities.

After experiments, SABLM-VD discovered 9 N-Day vulnerability instances in the binary functions of 4 open-source projects. The experimental results are shown in Table 16. In Table 16, we verified that the CVE numbers of 3 vulnerability instances are not in the NVD dataset, indicating that SABLM-VD independently detected 3 N-Day vulnerability instances outside the dataset. Among these 9 N-Day vulnerability instances, SABLM-VD detected 4, 3, 1, and 1 N-Day vulnerability instances from Qemu, FFmpeg, Libav, and Xen, respectively. These vulnerability numbers range from CVE-2011-3934 to CVE-2020-13253, covering a wide time span and coming from multiple open-source projects. Among them, 3 N-Day vulnerability instances outside the dataset were independently detected by SABLM-VD, which indicates that SABLM-VD is effective in detecting N-Day vulnerabilities in real-world codes.

Answer to RQ6: SABLM-VD is effective in detecting N-Day vulnerabilities in real-world codes.

Table 17

The time cost of different methods at each phase on the SARD (ARM) dataset.

Method	Data processing	Training	Detection
GNN-s2v _(att)	0.06 s/sample	0.78 h/epoch	0.05 s/sample
GNN-s2v _(lstm)	(Same as above)	0.16 h/epoch	0.006 s/sample
HAformer	1.28 s/sample	0.53 h/epoch	0.008 s/sample
SABLM-VD	1.74 s/sample	0.80 h/epoch	0.02 s/sample

5.7. Efficiency (RQ7)

To answer RQ7, we take the SARD (ARM) dataset with the largest sample size as an example and record the time cost of SABLM-VD and GNN-s2v_(att), GNN-s2v_(lstm), HAformer in the data processing phase, training phase, and detection phase, as shown in Table 17. Since the training epoch of SABLM-VD is 8 epochs, while GNN-s2v_(att) requires at least 10 epochs to achieve the best performance in practice, the total time cost of SABLM-VD in the training phase is lower than that of GNN-s2v_(att). However, the time cost of SABLM-VD in the data processing phase is higher than that of other methods, which is intuitive because decompiling assembly code to generate pseudocode consumes more computing resources. Nevertheless, considering that the detection performance of SABLM-VD is significantly better than the existing baselines, we believe that the additional time cost is worthwhile.

Answer to RQ7: SABLM-VD has a suitable detection efficiency, and it takes 0.02 s to detect a single sample.

6. Discussion

6.1. Effectiveness discussion

In previous studies [7,8,10,11], researchers often determined the detection results by similarity comparison and threshold setting, which made it difficult for the detection methods to identify vulnerabilities hidden in character-level differences; at the same time, it might misjudge benign codes that were slightly modified but overall similar to the samples in the vulnerability database. Moreover, previous studies ignored the semantic features of data transfer and arithmetic logic in assembly code, resulting in an incomplete and inaccurate learning of vulnerability patterns by the detection models. Therefore, we conducted research on the assembly instructions of x86-64 and ARM architectures and proposed SABLM-VD with binary code semantic awareness. Sections 5.3 and 5.4 discuss and validate the effectiveness of hybrid-granularity tokenization and the SABLM pre-training strategy. We believe that the SABLM pre-training strategy, by training the language model to predict the operand information of data transfer instructions, arithmetic logic instructions, and control flow instructions in assembly code, enhances the model's comprehensive understanding of assembly code semantics. Finally, we also integrated SABLM with pseudocode features, thereby enabling SABLM-VD to achieve significantly better performance than all baselines.

6.2. Case study

Through case studies, the effectiveness of SABLM-VD can be further verified and its semantic perception details demonstrated. To this end, we visualize the attention weights of the assembly semantic vectors generated in SABLM-VD. Specifically, we take the x86-64 architecture vulnerability function *init_sig_algs* (CVE-2021-3449 [53]) and the ARM architecture vulnerability function *aoev1_reserve_print* (CVE-2017-16808 [54]) from the NVD dataset as cases, and output their assembly semantic vector attention weights. Then, by combining the weights with the hybrid-granularity assembly codes, we generate the corresponding attention weight heatmaps, as shown in Fig. 14. It should be

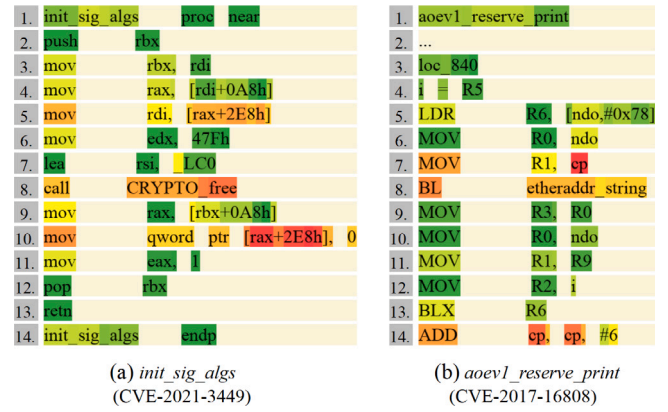


Fig. 14. Attention weight heatmaps for the assembly code of functions *init_sig_algs* and *aoev1_reserve_print*. The words/subwords with the highest weight have a red background color. As the weight decreases, the background color changes from red to yellow to green. The words/subwords with a green background have the lowest weight.

noted that these assembly codes are generated by IDA disassembly and differ slightly from those developed in the forward direction.

(1) The function *init_sig_algs* belongs to the *extensions.o* of OpenSSL 1.1.1 and is responsible for resetting the signature algorithm state. This function contains a CWE-476 (NULL Pointer Dereference) vulnerability numbered CVE-2021-3449. The vulnerability mechanism involves calling *CRYPTO_free* to release a pointer without synchronously clearing the corresponding length field, leading to a possible NULL pointer dereference. In Fig. 14(a), SABLM-VD notices the key control flow instruction opcode `call`, data transfer instruction opcode `mov`, pointer release-related operands `CRYPTO_free` and `[rax+2E8h]`, and gives the highest attention weight (red background) to the pointer address `[rax+2E8h]` when it is set to NULL, and gives relatively high attention weights (orange-red to orange background) to `call`, `mov` (lines 5 and 10), and `CRYPTO_free`.

(2) The function *aoev1_reserve_print* belongs to the *print-aoe.o* of Tcpdump 4.8.0 and contains a CWE-125 (Out-of-bounds Read) vulnerability numbered CVE-2017-16808. The vulnerability mechanism involves failing to check the length of pointer `cp` before calling *etheraddr_string* in a loop, while `cp` increments within the loop, potentially causing an out-of-bounds read. In Fig. 14(b), SABLM-VD notices the key control flow instruction opcode `BL`, arithmetic logic instruction opcode `ADD`, and operands `cp` and *etheraddr_string*, and gives the highest attention weight (red background) to `cp`, and relatively high attention weights (orange-red to orange background) to `BL`, `ADD`, and *etheraddr_string*.

Based on these two cases, we believe that SABLM-VD can effectively perceive the semantics of assembly code for both x86-64 and ARM architectures, and can capture code features and vulnerability patterns that contribute to vulnerability.

6.3. Limitations and future work

The limitations of SABLM-VD, along with our planned future work, are as follows. (1) There are challenges when dealing with the binary code of other architectures, such as MIPS, RISC-V, and PowerPC. The reason is that the method in this paper is architecture-dependent, and we merely selected the two most extensively utilized architectures for the design of the method. In future work, we will explore effective solutions for other binary architectures. (2) The performance is affected by the disassembler and decompiler. The reason is that the reverse analysis chain from binary machine code to assembly code and then to pseudocode is not completely accurate, and there may still be potential information loss or errors. In future work, we will consider including

the original machine code in the feature learning scope and alleviating such problems through feature fusion. (3) It is difficult to deal with cross-functional vulnerability types. The reason is that the method in this paper only focuses on whether a single function is vulnerable, and the pre-training strategy constructed in this way is difficult to apply to cross-function semantic learning and vulnerability detection. We plan to study this issue in future research.

7. Conclusion

In this paper, we conduct research on x86-64 and ARM architecture binaries and propose a cross-architecture binary code vulnerability detection framework, SABLM-VD. It consists of a BERT-based semantic-aware binary language model, SABLM, and a pseudocode embedding component. SABLM is a solution that simultaneously embeds the data transfer semantics, arithmetic logic semantics, and control flow of assembly instructions into a BERT-based language model, effectively perceiving the semantic features of assembly code. Additionally, to retain assembly information while alleviating the OOV problem, we propose a hybrid-granularity assembly language tokenization method. We conducted experiments on the NVD dataset and the SARD dataset and validated the effectiveness of SABLM-VD. Furthermore, we demonstrated the advantages of SABLM-VD through ablation studies, visualizations, and real-world applications. Compared with the state-of-the-art baseline methods, SABLM-VD has better performance and can detect more vulnerabilities.

CRedit authorship contribution statement

Qinghao Li: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Tieming Liu:** Resources, Funding acquisition. **Wei Liu:** Writing – review & editing. **Yonghe Tang:** Writing – review & editing. **Chunling Liu:** Writing – review & editing. **Weiyu Dong:** Writing – review & editing, Resources, Project administration, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the Key R&D Special Program of Henan Province (No. 221111210300).

Data availability

The source code supporting this study has been deposited in a public GitHub repository and is available at: <https://github.com/luckylili/SABLM-VD>.

References

- [1] Synopsys, Rep-ossra-2024, 2024, <https://www.synopsys.com/software-integrity/resources/analyst-reports/open-source-security-risk-analysis.html>.
- [2] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, Y. Zhong, VulDeePecker: A deep learning-based system for vulnerability detection, in: Proceedings 2018 Network and Distributed System Security Symposium, Internet Society, San Diego, CA, 2018.
- [3] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, Z. Chen, SySeVR: A framework for using deep learning to detect software vulnerabilities, IEEE Trans. Dependable Secur. Comput. 19 (4) (2022) 2244–2258.
- [4] S. Chakraborty, R. Krishna, Y. Ding, B. Ray, Deep learning based vulnerability detection: Are we there yet? IEEE Trans. Softw. Eng. 48 (9) (2022) 3280–3296.
- [5] M. Fu, C. Tantithamthavorn, LineVul: A transformer-based line-level vulnerability prediction, in: Proceedings of the 19th International Conference on Mining Software Repositories, in: MSR '22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 608–620.
- [6] W. Wang, T.N. Nguyen, S. Wang, Y. Li, J. Zhang, A. Yadavally, DeepVD: Toward class-separation features for neural network vulnerability detection, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), 2023, pp. 2249–2261.
- [7] X. Xu, C. Liu, Q. Feng, H. Yin, L. Song, D. Song, Neural network-based graph embedding for cross-platform binary code similarity detection, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, 2017, pp. 363–376.
- [8] J. Gao, X. Yang, Y. Fu, Y. Jiang, J. Sun, Vulseeker: A semantic learning based vulnerability seeker for cross-platform binary, in: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, 2018, pp. 896–899.
- [9] J. Gao, Y. Jiang, Z. Liu, X. Yang, C. Wang, X. Jiao, Z. Yang, J. Sun, Semantic learning and emulation based cross-platform binary vulnerability seeker, IEEE Trans. Softw. Eng. 47 (11) (2019) 2575–2589.
- [10] X. Li, Y. Qu, H. Yin, Palmtree: Learning an assembly language model for instruction embedding, in: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, 2021, pp. 3236–3251.
- [11] H. Wang, W. Qu, G. Katz, W. Zhu, Z. Gao, H. Qiu, J. Zhuge, C. Zhang, Jtrans: Jump-aware transformer for binary code similarity detection, in: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, 2022, pp. 1–13.
- [12] Z. Luo, P. Wang, B. Wang, Y. Tang, W. Xie, X. Zhou, D. Liu, K. Lu, VulHawk: Cross-architecture vulnerability detection with entropy-based binary code search, in: NDSS, 2023.
- [13] Y. Wang, P. Jia, X. Peng, C. Huang, J. Liu, BinVulDet: Detecting vulnerability in binary program via decompiled pseudo code and bilstm-attention, Comput. Secur. 125 (2023) 103023.
- [14] L. Li, S.H. Ding, Y. Tian, B.C. Fung, P. Charland, W. Ou, L. Song, C. Chen, VulANalyzeR: Explainable binary vulnerability detection with multi-task learning and attentional graph convolution, ACM Trans. Priv. Secur. 26 (3) (2023) 1–25.
- [15] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, in: Proceedings of the 2019 Conference of the North, Association for Computational Linguistics, 2019, pp. 4171–4186.
- [16] SARD, 2023, 2023, <https://samate.nist.gov/SRD/index.php>.
- [17] CVE, CVE list downloads, 2024, <https://www.cve.org/Downloads/>.
- [18] Hex-Rays, IDA pro disassembler and debugger, 2015, <https://www.hex-rays.com/products/ida/index.shtml>.
- [19] B. Saridou, J.R. Rose, S. Shiaeles, B.K. Papadopoulos, 48,240 malware samples and binary visualisation images for machine learning anomaly detection, 2021.
- [20] A. Dhavle, S. Shukla, A novel malware detection mechanism based on features extracted from converted malware binary images, 2021, arXiv:2104.06652.
- [21] X. Jiang, S. Wang, Y. Gong, T. Yu, L. Liu, X. Yu, HAFormer: Semantic fusion of hex machine code and assembly code for cross-architecture binary vulnerability detection, Comput. Secur. 145 (2024) 104029.
- [22] T. Le, T. Nguyen, T. Le, D.Q. Phung, P. Montague, O.Y. de Vel, L. Qu, Maximal divergence sequential autoencoder for binary software vulnerability detection, in: International Conference on Learning Representations, 2019.
- [23] H. Yan, S. Luo, L. Pan, Y. Zhang, HAN-BSVD: A hierarchical attention network for binary software vulnerability detection, Comput. Secur. 108 (2021) 102286.
- [24] K. Pei, Z. Xuan, J. Yang, S. Jana, B. Ray, Learning approximate execution semantics from traces for binary function similarity, IEEE Trans. Softw. Eng. 49 (4) (2023) 2776–2790.
- [25] J. Gao, X. Yang, Y. Fu, Y. Jiang, H. Shi, J. Sun, VulSeeker-pro: Enhanced semantic learning based binary vulnerability seeker with Emulation, in: Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, in: ESEC/FSE 2018, Association for Computing Machinery, New York, NY, USA, 2018, pp. 803–808.
- [26] S.H.H. Ding, B.C.M. Fung, P. Charland, Asm2Vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler optimization, in: 2019 IEEE Symposium on Security and Privacy (SP), IEEE, San Francisco, CA, USA, 2019, pp. 472–489.
- [27] Y. Duan, X. Li, J. Wang, H. Yin, DeepBinDiff: Learning program-wide code representations for binary diffing, in: 27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23–26, 2020, The Internet Society, 2020.
- [28] B. Liu, W. Huo, C. Zhang, W. Li, F. Li, A. Piao, W. Zou, ADiff: Cross-Version binary code similarity detection with DNN, in: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, in: ASE '18, Association for Computing Machinery, New York, NY, USA, 2018, pp. 667–678.
- [29] S. Yang, L. Cheng, Y. Zeng, Z. Lang, H. Zhu, Z. Shi, Asteria: Deep learning-based AST-encoding for cross-platform binary code similarity detection, in: 2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), 2021, pp. 224–236.

- [30] S. Lipp, S. Banescu, A. Pretschner, An empirical study on the effectiveness of static C code analyzers for vulnerability detection, in: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, in: ISSTA 2022, Association for Computing Machinery, New York, NY, USA, 2022, pp. 544–555.
- [31] Z. Zhang, M. Sabuncu, Generalized cross entropy loss for training deep neural networks with noisy labels, in: S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, R. Garnett (Eds.), *Advances in Neural Information Processing Systems*, 31, Curran Associates, Inc., 2018.
- [32] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, M. Zhou, CodeBERT: A pre-trained model for programming and natural languages, in: Findings of the Association for Computational Linguistics: EMNLP 2020, in: Findings of ACL, EMNLP 2020, Association for Computational Linguistics, 2020, pp. 1536–1547.
- [33] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, J. Yin, UniXcoder: Unified cross-modal pre-training for code representation, in: S. Muresan, P. Nakov, A. Villavicencio (Eds.), *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2022, Dublin, Ireland, May 22–27, 2022, Association for Computational Linguistics, 2022, pp. 7212–7225.
- [34] H. Hanif, S. Maffei, VulBERTa: Simplified source code pre-training for vulnerability detection, in: 2022 International Joint Conference on Neural Networks (IJCNN), 2022, pp. 1–8.
- [35] Vector35, Binary ninja, 2025, <https://binary.ninja/>.
- [36] Intel, X86 instructions, 2025, <https://cdrdv2.intel.com/v1/dl/getContent/671200>.
- [37] Microsoft, X64 instructions, 2024, <https://learn.microsoft.com/en-us/windows-hardware/drivers/debugger/x64-instructions>.
- [38] Microsoft, X86 instructions, 2024, <https://learn.microsoft.com/en-us/windows-hardware/drivers/debugger/x86-instructions>.
- [39] ARM, Aarch32 – base instructions, 2024, <https://developer.arm.com/documentation/ddi0597/2024-12/Base-Instructions?lang=en>.
- [40] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.
- [41] Z. Zhang, Improved Adam optimizer for deep neural networks, in: 2018 IEEE/ACM 26th International Symposium on Quality of Service (IWQoS), 2018, pp. 1–2.
- [42] Y. Li, D. Tarlow, M. Brockschmidt, R. Zemel, Gated graph sequence neural networks, in: Y. Bengio, Y. LeCun (Eds.), 4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2–4, 2016, Conference Track Proceedings, 2016.
- [43] Y. Li, C. Gu, T. Dullien, O. Vinyals, P. Kohli, Graph matching networks for learning the similarity of graph structured objects, in: K. Chaudhuri, R. Salakhutdinov (Eds.), *Proceedings of the 36th International Conference on Machine Learning*, in: *Proceedings of Machine Learning Research*, vol. 97, PMLR, 2019, pp. 3835–3845.
- [44] L. Massarelli, G.A. Di Luna, F. Petroni, L. Querzoni, R. Baldoni, et al., Investigating graph embedding neural networks with unsupervised features extraction for binary analysis, in: *Proceedings of the 2nd Workshop on Binary Analysis Research (BAR)*, 2019, pp. 1–11.
- [45] L. Massarelli, G.A. Di Luna, F. Petroni, R. Baldoni, L. Querzoni, SAFE: Self-attentive function embeddings for binary similarity, in: R. Perdisci, C. Maurice, G. Giacinto, M. Almgren (Eds.), *Detection of Intrusions and Malware, and Vulnerability Assessment*, Springer International Publishing, Cham, 2019, pp. 309–329.
- [46] H. Wang, Z. Gao, C. Zhang, Z. Sha, M. Sun, Y. Zhou, W. Zhu, W. Sun, H. Qiu, X. Xiao, CLAP: Learning transferable binary code representations with natural language supervision, in: *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, in: Issta 2024, Association for Computing Machinery, New York, NY, USA, 2024, pp. 503–515.
- [47] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, et al., The Llama 3 herd of models, 2024, CoRR [arXiv:2407.21783](https://arxiv.org/abs/2407.21783). [arXiv:2407.21783](https://arxiv.org/abs/2407.21783).
- [48] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, et al., Deepseek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025, arXiv preprint [arXiv:2501.12948](https://arxiv.org/abs/2501.12948). [arXiv:2501.12948](https://arxiv.org/abs/2501.12948).
- [49] H. Dai, B. Dai, L. Song, Discriminative embeddings of latent variable models for structured data, in: M.F. Balcan, K.Q. Weinberger (Eds.), *Proceedings of the 33rd International Conference on Machine Learning*, in: *Proceedings of Machine Learning Research*, vol. 48, PMLR, New York, New York, USA, 2016, pp. 2702–2711.
- [50] L. van der Maaten, G. Hinton, Visualizing Data using T-SNE, *J. Mach. Learn. Res.* 9 (86) (2008) 2579–2605.
- [51] C. Mao, Z. Zhong, J. Yang, C. Vondrick, B. Ray, Metric learning for adversarial robustness, in: H. Wallach, H. Larochelle, A. Beygelzimer, F. d’Alché-Buc, E. Fox, R. Garnett (Eds.), *Advances in Neural Information Processing Systems*, vol. 32, Curran Associates, Inc., 2019.
- [52] P. Junod, J. Rinaldini, J. Wehrli, J. Michielin, Obfuscator-LLVM – software protection for the masses, in: 2015 IEEE/ACM 1st International Workshop on Software Protection, 2015, pp. 3–9.
- [53] cvedetails, CVE-2021-3449, 2021, <https://www.cvedetails.com/cve/CVE-2021-3449/>.
- [54] cvedetails, CVE-2017-16808, 2017, <https://www.cvedetails.com/cve/CVE-2017-16808/>.